

# Weighted Batch Threshold Encryption

No Author Given

No Institute Given

**Abstract.** Batch threshold encryption allows a committee to decrypt a selected batch of ciphertexts using one short pre-decryption key per party, while ciphertexts outside the batch remain private. This makes batch threshold encryption attractive for building encrypted mempools for blockchains, where validators reveal transactions selected for a block while pending transactions remain private. However, in proof-of-stake systems such as Solana or Ethereum, authorization depends on each validator’s stake, which motivates a weighted version of batch threshold encryption. Naive approaches to weighting existing batch threshold encryption schemes lead to communication costs proportional to the weight assigned to each party. Existing constructions to solve this problem are either not weighted, or require having some sort of labels or indices for ciphertexts.

We present the first weighted batch threshold encryption schemes that require neither batch/epoch labels nor encryption-time ciphertext indices, while keeping each party’s communication independent of its weight. We obtain two constructions by virtualizing prior schemes: the **BTX** construction of Agarwal et al. and the partial fraction construction of Boneh et al. In both schemes’ decryption procedures, each real party communicates a single group element for an entire batch, regardless of its assigned weight. We prove correctness, robustness, and security under weighted variants of the source assumptions, and analyze these assumptions in generic bilinear groups. For the more efficient **BTX**-based construction, we also give a specialized distributed key generation protocol that avoids a general purpose MPC-based setup, as required by the previous state of the art schemes. Finally, we implement our schemes in Rust to demonstrate their practical performance at parameter sizes relevant to encrypted mempool.

**Keywords:** Threshold encryption · Batch encryption · Bilinear groups

## 1 Introduction

## 2 Preliminaries

### 2.1 Basic preliminaries

**Bilinear groups.** Let  $(p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, [1]_1, [1]_2, e) \leftarrow \text{GrpGen}(1^\lambda)$  be the descriptions of prime-order groups, their identity elements, and a nondegenerate bilinear

pairing  $e$ . We use additive notation for group operations, so  $e([a]_1, [b]_2) = [ab]_T$ . All algorithms implicitly receive the group descriptions, and all scalar arithmetic is over  $\mathbb{F}_p$ .

**Weights and sharing.** Let  $N$  denote the number of real parties in our scheme, and  $W$  denote the total weight assigned to the  $N$  parties. Party  $j$  has positive integer weight  $w_j$  and owns a set  $\Omega_j \subset \mathbb{F}_p^*$  of  $w_j$  evaluation points used for Shamir secret sharing. The sets  $(\Omega_j)_{j \in [N]}$  are disjoint. For a set of parties  $T$ , set the notation

$$\Omega_T = \bigcup_{j \in T} \Omega_j, \quad \text{wt}(T) = |\Omega_T| = \sum_{j \in T} w_j, \quad W = \text{wt}([N]) = |\Omega_{[N]}|.$$

For a corruption bound  $t < W$ , a set  $T$  is authorized if  $\text{wt}(T) > t$  and unauthorized if  $\text{wt}(T) \leq t$ . For authorized  $T$ , let  $\lambda_{T,\omega}$  be the Lagrange coefficient at zero for each  $\omega \in \Omega_T$  over interpolation set  $\Omega_T$ .<sup>1</sup> For every polynomial  $F$  of degree at most  $t$ ,

$$F(0) = \sum_{\omega \in \Omega_T} \lambda_{T,\omega} F(\omega).$$

The party which owns a share  $\omega \in \Omega_j$  is denoted by  $\text{own}(\omega) = j$ .

## 2.2 Weighted batch threshold encryption

We now define the syntax and security properties of our scheme. The following syntax extends the batch threshold encryption definition of Boneh et al. [12] to a weighted access structure.

**Definition 1 (Weighted BTE syntax).** *A weighted batch threshold encryption scheme WBTX with maximum batch size  $\ell$ , total weight  $W$ , threshold  $t$ , and message space  $\mathcal{M}$  consists of the following PPT algorithms:*

- $\text{Setup}(1^\lambda, 1^\ell, 1^W, 1^t, (w_j)_{j \in [N]}) \rightarrow (\text{pp}, \text{ek}, \text{dk}, (\text{sk}_j, \text{vk}_j)_{j \in [N]}):$  Given security parameter  $\lambda$ , maximum batch size  $\ell$ , total weight  $W$ , threshold  $t$  and per-party weights  $w_j$ , outputs public parameters  $\text{pp}$ , a public encryption key  $\text{ek}$ , public decryption key  $\text{dk}$ , and per-party secret/verification keys  $(\text{sk}_j, \text{vk}_j)_{j \in [N]}$ .  $\text{pp}$  is an implicit input to the subsequent algorithms.
- $\text{Enc}(\text{ek}, m) \rightarrow \text{ct}: \text{encrypts } m \text{ using } \text{ek}.$
- $\text{PreDec}(\text{sk}_j, (\text{ct}_i)_{i \in [\ell]}) \rightarrow \text{sbk}_j \text{ or } \perp:$  Using a party's secret key  $\text{sk}_j$ , and a set of up to  $\ell$  ciphertexts, outputs the corresponding **short** pre-decryption key  $\text{sbk}_j$  per party.
- $\text{PreDecVer}(\text{vk}_j, (\text{ct}_i)_{i \in [\ell]}, \text{sbk}_j) \rightarrow 1 \text{ or } 0:$  Using a party's verification key  $\text{vk}_j$ , a set of up to  $\ell$  ciphertexts, and a claimed pre-decryption key  $\text{sbk}_j$ , outputs 1 if the pre-decryption key is the correct key corresponding to the batch of ciphertexts, and 0 otherwise.

<sup>1</sup> i.e.  $\lambda_{T,\omega}$  is the Lagrange indicator function  $\prod_{u \in \Omega_T, u \neq \omega} \frac{X-u}{\omega-u}$  evaluated at  $X = 0$ .

- $\text{Dec}(\text{dk}, T, (\text{sbk}_j)_{j \in T}, (\text{ct}_i)_{i \in [\ell]}) \longrightarrow (m_i)_{i \in [\ell]}$  or  $\perp$ : decrypts a batch of ciphertexts  $(\text{ct}_i)_{i \in [\ell]}$  using a collection of pre-decryption keys  $(\text{sbk}_j)_{j \in T}$  corresponding to a set of parties  $T$  with total weight greater than  $t$ . Decryption first verifies each ciphertext and submitted keys. It may discard invalid keys, but it can only return messages if the accepting set is authorized.

**Definition 2 (Correctness).** A weighted BTE scheme is correct if, for every authorized  $T$ , every message vector  $(m_i)_{i \in [\ell]}$ , honestly generated public parameters and secret keys, and honest pre-decryption keys  $\text{sbk}_j \leftarrow \text{PreDec}(\text{sk}_j, (\text{ct}_i)_{i \in [\ell]})$  for all  $j \in T$ , decryption returns  $(m_i)_{i \in [\ell]}$ , except with negligible probability.

We adapt the static batch chosen-ciphertext game of Boneh et al. [12] to this context by replacing cardinality with weight. The formulation below gives the adversary all honest pre-decryption keys, which is at least as strong as giving it the final decryption output.

**Definition 3 (Static WB-IND-CCA security).** The security game, parameterized by  $\ell$ ,  $N$ ,  $W$  and  $t$ , is as follows:

1. The challenger generates keys/parameters via **Setup** and gives  $(\text{ek}, \text{dk})$  to  $\mathcal{A}$ . The adversary then fixes an (unauthorized) corruption set  $\mathcal{C} \subseteq [N]$  with  $\text{wt}(\mathcal{C}) \leq t$  and receives  $(\text{sk}_j)_{j \in \mathcal{C}}$ .
2.  $\mathcal{A}$  may submit batches of at most  $\ell$  ciphertexts. For each batch, the challenger returns  $\text{PreDec}(\text{sk}_j, \cdot)$  for every  $j \notin \mathcal{C}$ .
3. The adversary submits  $m_0, m_1 \in \mathcal{M}$  of equal length. The challenger samples  $b \leftarrow \{0, 1\}$  and returns  $\text{ct}^* \leftarrow \text{Enc}(\text{ek}, m_b)$ .
4.  $\mathcal{A}$  may submit batches of at most  $\ell$  ciphertexts which do not contain  $\text{ct}^*$ . For each batch, the challenger returns  $\text{PreDec}(\text{sk}_j, \cdot)$  for every  $j \notin \mathcal{C}$ .
5.  $\mathcal{A}$  outputs bit  $b'$ .

The advantage of  $\mathcal{A}$  against the weighted BTE scheme WBTX is

$$\text{Adv}_{\text{WBTX}, \mathcal{A}, \ell, N, W, t}^{\text{WB-IND-CCA}}(\lambda) = |\Pr[b' = b] - \frac{1}{2}|.$$

The scheme is secure if this advantage is negligible for every PPT adversary  $\mathcal{A}$  and polynomial  $\ell, N, W, t$ .

A weighted BTE scheme must be *efficient* in that (i) the size of the pre-decryption keys  $(\text{sbk}_j)_{j \in [N]}$  is succinct in the maximum batch size  $\ell$  and weight assigned to party  $j$ , and (ii) the size of the ciphertexts is succinct in the number of parties.

### 2.3 Robustness

A weighted batch threshold encryption scheme satisfies share robustness if it has accurate share verification and accurate blaming. We adapt the definitions of these properties from [13, Definitions 22.13 & 22.14].

**Definition 4 (Accurate blaming).** *Let*

$$\text{Blame}(\text{vk}, T, (\text{sbk}_j)_{j \in T}, (\text{ct}_i)_{i \in [\ell]}) = \{j \in [T] : \text{PreDecVer}(\text{vk}_j, (\text{ct}_i)_{i \in [\ell]}, \text{sbk}_j) = 0\}$$

*denote the set of parties whose pre-decryption shares are rejected by the pre-decryption share verification algorithm. A weighted BTE scheme ensures accurate blaming if, for every honestly generated parameter tuple, every ciphertext batch  $(\text{ct}_i)_{i \in [\ell]}$ , every authorized submitted set  $T \subseteq [N]$ , every collection of shares  $(\text{sbk}_j)_{j \in T}$ , and every  $j \in T$ , membership  $j \in \text{Blame}(\text{vk}, T, (\text{sbk}_j)_{j \in T}, (\text{ct}_i)_{i \in [\ell]})$  implies*

$$\Pr [\text{PreDec}(\text{sk}_j, (\text{ct}_i)_{i \in [\ell]}) = \text{sbk}_j] = 0,$$

*where the probability is over the internal randomness of PreDec.*

**Definition 5 (Consistency).** *We define the following security game:*

1. *The challenger invokes  $(\text{pp}, \text{ek}, \text{dk}, (\text{sk}_j, \text{vk}_j)_{j \in [N]}) \leftarrow \text{Setup}(1^\lambda, 1^\ell, 1^W, 1^t, (w_j)_{j \in [N]})$  and returns the output to Adv.*
2. *Adv sends  $((\text{ct}_i)_{i \in [\ell]}, S_1, \{\text{sbk}_j^{(1)}\}_{j \in S_1}, S_2, \{\text{sbk}_j^{(2)}\}_{j \in S_2})$ , where  $(\text{ct}_i)_{i \in [\ell]}$  is a batch of  $\ell$  ciphertexts,  $S_1, S_2 \subseteq [N]$  are (authorized) sets of parties with weight more than  $t$ , and  $\{\text{sbk}_j^{(1)}\}_{j \in S_1}$  and  $\{\text{sbk}_j^{(2)}\}_{j \in S_2}$  are pre-decryption shares.*
3. *Define  $\text{Dec}(\text{dk}, S_1, (\text{sbk}_j)_{j \in S_1}, (\text{ct}_i)_{i \in [\ell]}) = (m_i^{(1)})_{i \in [\ell]}$  and  $\text{Dec}(\text{dk}, S_2, (\text{sbk}_j)_{j \in S_2}, (\text{ct}_i)_{i \in [\ell]}) = (m_i^{(2)})_{i \in [\ell]}$ . Adv wins the game if  $(m_i^{(1)})_{i \in [\ell]} \neq \perp$ ,  $(m_i^{(2)})_{i \in [\ell]} \neq \perp$ , and  $(m_i^{(1)})_{i \in [\ell]} \neq (m_i^{(2)})_{i \in [\ell]}$ .*

*The advantage of Adv, denoted by  $\text{Adv}_{\text{WBTX}, \mathcal{A}, \ell, N, W}^{\text{Cons}}$  is the probability that it wins the game. The WBTX scheme is said to be consistent if this advantage is negligible for every PPT adversary, and polynomial  $\ell, N, W$ .*

## 2.4 Non-Interactive Zero-Knowledge Proofs (NIZKs)

Both constructions use an online simulation (and straightline)-extractable NIZK  $\Pi_{\text{DL}}$ . For the group element  $C \in \mathbb{G}_1 \times \mathbb{G}_T$ , it proves the relation

$$R_{\text{DL}} = \{(C, r) : C = [r]_1\}.$$

It allows the extractor in our proofs to recover the scalar values needed to simulate a pre-decryption key for a given ciphertext. We also require  $\Pi_{\text{DL}}$  to be complete and zero-knowledge as defined below.

**Definition 6 (Non-Interactive Zero Knowledge Proofs).** *Let  $\mathcal{R} \subseteq \mathcal{X} \times \mathcal{W}$  be a relation over sets  $\mathcal{X}$  and  $\mathcal{W}$ . A NIZK scheme NIZK for  $\mathcal{R}$ <sup>2</sup> is a tuple of PPT algorithms,  $\text{NIZK} = (\text{Setup}, \text{Prove}, \text{Verify})$  defined as follows.*

<sup>2</sup> Later, we will instead have a family of indexed relations to capture public parameters with which the relation is specified.

- $\text{Setup}(1^\lambda) \rightarrow \text{crs}$ : given the security parameter  $\lambda$ , outputs a common reference string  $\text{crs}$ . The algorithms below implicitly take  $\text{crs}$  as input.
- $\text{Prove}(x, w) \rightarrow \text{pf}$ : given a statement  $x \in \mathcal{X}$  and a witness  $w \in \mathcal{W}$ , outputs a proof  $\text{pf}$ .
- $\text{Verify}(x, \text{pf}) \rightarrow 0/1$ : given a statement  $x \in \mathcal{X}$  and a proof  $\text{pf}$ , outputs 0 or 1.

**Definition 7 (Completeness).** A NIZK scheme for  $\mathcal{R}$  is complete if for all  $(x, w) \in \mathcal{R}$ ,

$$\Pr[\text{Verify}(x, \pi) = 1 : \pi \leftarrow \text{Prove}(x, w)] \geq 1 - \text{negl}(\lambda)$$

**Definition 8 (Zero-Knowledge).** A NIZK scheme for  $\mathcal{R}$  is zero-knowledge if there exists a PPT simulator  $\text{Sim}$  such that for all  $(x, w) \in \mathcal{R}$ ,  $(x, \text{Sim}(x))$  is computationally indistinguishable from  $(x, \text{Prove}(x, w))$ .

Simulation extractability states that after seeing simulated proofs, every fresh accepting statement–proof pair has an extractable witness, except with negligible probability.

**Definition 9 (Simulation extractability [16]).** A NIZK scheme for  $\mathcal{R}$ ,  $\text{NIZK} = (\text{Setup}, \text{Prove}, \text{Verify})$ , is said to be simulation extractable or an SE-NIZK if there exists a PPT extractor  $\text{Ext}$  and a PPT simulator  $\text{Sim} = (\text{SimSetup}, \text{SimProve})$  such that for all PPT adversaries  $\text{Adv}$ , there exists a negligible function  $\text{negl}(\lambda)$  for which the following holds:

$$\Pr \left[ \begin{array}{l} \text{Verify}(x^*, \pi^*) = 1 \wedge \\ x^* \notin Q \wedge \\ (x^*, w^*) \notin \mathcal{R} \end{array} \middle| \begin{array}{l} (\text{crs}, \text{td}_{\text{Sim}}, \text{td}_{\text{Ext}}) \leftarrow \text{SimSetup}(1^\lambda); \\ Q \leftarrow \emptyset; \\ (x^*, \pi^*) \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Sim}}}(\text{crs}); \\ w^* \leftarrow \text{Ext}(\text{crs}, \text{td}_{\text{Ext}}, x^*, \pi^*) \end{array} \right] \leq \text{negl}(\lambda).$$

Here,  $\mathcal{O}_{\text{Sim}}$  denotes the simulation oracle:

$$\mathcal{O}_{\text{Sim}}(x) : Q \leftarrow Q \cup \{x\}; \text{ return } \pi \leftarrow \text{SimProve}(\text{crs}, \text{td}_{\text{Sim}}, x)$$

As noted, in our scheme, we will also require the extractor to **straightline** (or online), which means that the extractor does not rewind the adversary.

### 3 Weighted BTX

In Construction 1, we present a weighted batch threshold encryption scheme, WBTX, based on ideas from [10,3,4]. To support weights, **Setup** samples an encryption key, secret shares the key using a degree- $t$  polynomial, and assigns secret shares to different parties according to their weights. **PreDec** allows one pre-decryption key to correspond to multiple evaluations of the polynomial sampled at setup. For batch size  $\ell$ , total weight  $W$  and  $N$  parties, the scheme's keys consist of one  $\mathbb{G}_T$  element for the encryption key,  $(2\ell - 1)W$   $\mathbb{G}_2$  elements for the decryption keys, and  $\ell N$   $\mathbb{G}_2$  elements for the verification keys.

[**Aditi:** Nusret: what if each  $\text{Pi}$  publishes dec key for a different batch of CTs? can we still decrypt CTs for which we have enough dec keys?] [**Nusret:** Correct, I added a remark about this.] [**Sasha:** It looks like the “overlapping batches” comment is only on the partial fractions scheme but not the BTX one.] [**Nusret:** Right, it applies to both schemes, so I brought it to this section.]

As in  $\text{BEAT}^{++}$  [1] and BTX [4], we can accelerate the decryption algorithm using the Fast Fourier transform (FFT). For each  $i \in [\ell]$ , the terms

$$\sum_{j \in T'} e \left( \text{sbk}_j, \sum_{w \in \Omega_j} \lambda_{T',w} \text{dk}_{w,-i} \right)$$

can be computed using one multi-pairing of arity  $|T'|$ . On the other hand, for every nonzero offset  $d \in \{-(\ell-1), \dots, \ell-1\}$ , define

$$D_d := \sum_{\omega \in \Omega_{T'}} \lambda_{T',\omega} \text{dk}_{\omega,d}, \quad D_0 := 0.$$

The remaining terms are

$$B_i := \sum_{\substack{k \in [\ell] \\ k \neq i}} e(\text{ct}_k[1], D_{k-i}), \quad i \in [\ell],$$

which form a middle product between the  $\mathbb{G}_1$ -valued sequence  $(\text{ct}_k[1])_k$  and the offset-indexed  $\mathbb{G}_2$ -valued sequence  $(D_d)_d$ . More precisely, during decryption, we reverse the  $D_d$ -sequence and zero-pad both sequences to length  $m := 2^{\lceil \log_2(2\ell) \rceil}$ . A  $\mathbb{G}_1$ -FFT of the ciphertext sequence and a  $\mathbb{G}_2$ -FFT of the reversed decryption key sequence are then paired coordinate-wise; a truncated inverse  $\mathbb{G}_T$ -FFT returns all  $(B_i)_{i \in [\ell]}$  simultaneously. Consequently, decryption uses  $\ell$  multi-pairings of arity  $|T'|$  and  $m$  ordinary pairings. When  $\ell$  is a power of two,  $m = 2\ell$ , giving an amortized cost of  $|T'| + 2$  pairing inputs per ciphertext.

Finally, note that the submitted pre-decryption keys can also be batch verified by sampling  $\alpha_j \leftarrow \mathbb{F}_p$  for  $j \in T$ , and checking

$$e \left( \sum_{j \in T} \alpha_j \text{sbk}_j, [1]_2 \right) = \sum_{i=1}^{\ell} e \left( \text{ct}_i[1], \sum_{j \in T} \alpha_j \text{vk}_{j,i} \right)$$

[**Sasha:** Which terms are FFT'ed here? I feel like this paragraph needs one more sentence, since it seems like the  $\mathbb{G}_2$  operations would be amenable to an FFT.] [**Nusret:** Addressed.]

*Remark 1 (Pre-decryption keys for overlapping batches).* The parties need not form their pre-decryption keys for the same batch. Let  $I_j \subseteq [\ell]$  denote the indices used by party  $j$ , and write  $\text{ct}_{j,k}$  for the ciphertext assigned index  $k$  by that party. Fix a ciphertext  $\text{ct}_i$  and let  $T_i$  be a set of parties whose batches contain this ciphertext at the same index  $i$ . If  $T_i$  is authorized, then  $\text{ct}_i$  can be decrypted

**Setup**( $1^\lambda, 1^\ell, 1^W, t, (w_j)_{j \in [N]}$ )  $\rightarrow$  ( $\text{pp}, \text{ek}, \text{dk}, (\text{sk}_j, \text{vk}_j)_{j \in [N]}$ ):

- ▷  $\text{crs}_{\Pi_{\text{DL}}} \leftarrow \Pi_{\text{DL}}.\text{Setup}(1^\lambda)$ .
- ▷ Sample  $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, [1]_1, [1]_2, [1]_T, p, e) \leftarrow \text{GrpGen}(1^\lambda)$ .
- ▷ Choose  $\Omega_{[N]} \subseteq \mathbb{F}_p$  with  $|\Omega_{[N]}| = W$  as an evaluation domain for secret sharing.
- ▷ Partition  $\Omega_{[N]}$  into disjoint  $(\Omega_j)_{j \in [N]}$  such that  $|\Omega_j| = w_j$  for all  $j \in [N]$ .
- ▷ Let  $\text{pp}$  be  $((\Omega_j)_{j \in [N]}, \text{crs}_{\Pi_{\text{DL}}}, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, [1]_1, [1]_2, [1]_T, p, e)$ .
- ▷ Sample  $z \leftarrow \mathbb{F}_p$  and a uniformly random polynomial  $Z(X) \in \mathbb{F}_p[X]$  of degree at most  $t$  such that  $Z(0) = z$ . Set  $z_\omega := Z(\omega)$  for  $\omega \in \Omega_{[N]}$  and  $\text{ek} := [z]_T$ .
- ▷ For all  $j \in [N]$ , sample  $q_j \leftarrow \mathbb{F}_p^*$ . Let  $\text{sk}_j := q_j$  denote party  $j$ 's secret key and  $\text{vk}_j := (\text{vk}_{j,i} := [(q_j)^i]_2)_{i \in [\ell]}$  be their verification key.
- ▷ Set  $\text{dk}_{\omega,d} := [z_\omega q_{\text{down}(\omega)}^d]_2$  for all  $d \in \{-\ell, \dots, -1, 1, \dots, \ell-1\}, \omega \in \Omega_{[N]}$ .
- ▷ Output  $(\text{pp}, \text{ek}, \text{dk} := (\text{dk}_{\omega,d})_{d \in \{-\ell, \dots, -1, 1, \dots, \ell-1\}, \omega \in \Omega_{[N]}}, (\text{sk}_j, \text{vk}_j)_{j \in [N]})$ .

**Enc**( $\text{ek}, m$ )  $\rightarrow$   $\text{ct}$ :

- ▷ Given  $m \in \mathbb{G}_T$ , sample  $r \leftarrow \mathbb{F}_p$ , set  $\text{ct}[1] := [r]_1$  and  $\text{ct}[2] := m + r \cdot \text{ek}$ .
- ▷ Obtain  $\pi \leftarrow \Pi_{\text{DL}}.\text{Prove}((\text{ct}[1], \text{ct}[2]); r)$ .
- ▷ Output  $\text{ct} := (\text{ct}[1], \text{ct}[2], \pi)$ .

**PreDec**( $\text{sk}_j, (\text{ct}_i)_{i \in [\ell]}$ )  $\rightarrow$   $\text{sbk}_j$ :

- ▷ Parsing  $\text{ct}_i = (\text{ct}_i[1], \text{ct}_i[2], \pi_i)$ , if any  $\Pi_{\text{DL}}.\text{Verify}((\text{ct}_i[1], \text{ct}_i[2]), \pi_i) = 0$ , return  $\perp$ .
- ▷ Otherwise, return  $\text{sbk}_j := \sum_{i=1}^\ell (q_j)^i \cdot \text{ct}_i[1] \in \mathbb{G}_1$ .

**PreDecVer**( $\text{vk}_j, (\text{ct}_i)_{i \in [\ell]}, \text{sbk}_j$ )  $\rightarrow$  0/1:

- ▷ Check  $e(\text{sbk}_j, [1]_2) = \sum_{i=1}^\ell e(\text{ct}_i[1], \text{vk}_{j,i})$ , and if so, return 1.

**Dec**( $\text{dk}, \{\text{sbk}_j\}_{j \in T}, (\text{ct}_i)_{i \in [\ell]}$ )  $\rightarrow$   $(m_i)_{i \in [\ell]}$ :

- ▷ Verify the pre-decryption keys  $\{\text{sbk}_j\}_{j \in T}$  by checking  $\text{PreDecVer}(\text{vk}_j, (\text{ct}_i)_{i \in [\ell]}, \text{sbk}_j)$  for every  $j \in T$ .
- ▷ Discard all pre-decryption keys which do not pass verification. Let  $T'$  be an authorized set of accepted shares. Return  $\perp$  if  $T'$  no longer has sufficient weight.
- ▷ Let  $\text{dk}_{\omega,0} := 0$  for all  $\omega \in \Omega_{[N]}$ .
- ▷ For each  $i \in [\ell]$ , output

$$m_i = \text{ct}_i[2] - \sum_{j \in T'} e \left( \text{sbk}_j, \sum_{\omega \in \Omega_j} \lambda_{T', \omega} \text{dk}_{\omega, -i} \right) + \sum_{\substack{k \in [\ell] \\ k \neq i}} e \left( \text{ct}_k[1], \sum_{\omega \in \Omega_{T'}} \lambda_{T', \omega} \text{dk}_{\omega, k-i} \right)$$

**Construction 1:** Weighted BTX construction.

from their pre-decryption keys after verifying each  $\text{sbk}_j$  against its own indexed batch  $(\text{ct}_{j,k})_{k \in I_j}$ . Indeed, in Construction 1, we can rewrite the decryption pad

as a sum over  $j \in T_i$  and, for the term belonging to  $j$ , subtract only the cross terms indexed by  $I_j \setminus \{i\}$ . The contribution of party  $j$  then reduces to

$$r_i \sum_{\omega \in \Omega_j} \lambda_{T_i, \omega} z_\omega,$$

and summing over  $T_i$  gives  $r_i z$ . For Construction 5, we can re-write  $L_{j,i}$  by replacing  $[\ell]$  in its two inner sums with  $I_j$  and using  $\text{ct}_{j,k}[1]$ . Equation (11) cancels the additional ciphertexts in party  $j$ 's batch, leaving the same  $g_{j,i}^2 r_i$  term as before; the remainder of the correctness proof is unchanged. Consequently, if one party's batch is a subset of another's and their common ciphertexts retain the same indices, every ciphertext in the smaller batch can be decrypted whenever the parties contributing a key for it have sufficient total weight.

**Theorem 1 (Correctness and robustness).** *Construction 1 is correct and robust.*

*Proof.* Correctness follows by inspection. Consider the second and third terms in the decryption equation:

$$\begin{aligned} & \sum_{j \in T'} e \left( \text{sbk}_j, \sum_{w \in \Omega_j} \lambda_{T', w} \text{dk}_{w, -i} \right) - \sum_{\substack{k \in [\ell] \\ k \neq i}} e \left( \text{ct}_k[1], \sum_{w \in \Omega'_T} \lambda_{T', w} \text{dk}_{w, k-i} \right) \\ &= \sum_{j \in T'} e \left( \sum_{k \in [\ell]} (q_j)^k \cdot [r_k]_1, \sum_{w \in \Omega_j} \lambda_{T', w} [z_w q_j^{-i}]_2 \right) - \sum_{\substack{k \in [\ell] \\ k \neq i}} e \left( [r_k]_1, \sum_{j \in T'} \sum_{w \in \Omega_j} \lambda_{T', w} [z_w (q_j)^{k-i}]_2 \right) \\ &= \left( \sum_{j \in T'} \sum_{w \in \Omega_j} \lambda_{T', w} z_w \right) [r_i]_T = r_i [z]_T = r_i \cdot \text{ek} \end{aligned}$$

The sum simplifies to the ciphertext pad  $r_i \cdot \text{ek}$ , and  $m_i = \text{ct}_i[2] - r_i \cdot \text{ek}$ .

Next, we consider robustness. To show accurate blaming, observe that a correctly generated pre-decryption key always passes its verification check: **[Sasha: Would it be correct to refactor the third sum in this equation to have the inside of the sum be  $e((q_j)^i \cdot \text{ct}_i[1], [1]_2)$ ? that looks more right to me]** **[Nusret: Ah, right, fixed.]**

$$e(\text{sbk}_j, [1]_2) = e \left( \sum_{i=1}^{\ell} (q_j)^i \cdot \text{ct}_i[1], [1]_2 \right) = \sum_{i=1}^{\ell} e(\text{ct}_i[1], (q_j)^i) = \sum_{i=1}^{\ell} e(\text{ct}_i[1], \text{vk}_{j,i}) \quad (1)$$

Finally, by Equation (1), every accepted set of pre-decryption keys  $(\text{sbk}_j)_{j \in T}$ , for which the decryption algorithm outputs a non- $\perp$  value satisfies  $\text{sbk}_j = \sum_{i=1}^{\ell} (q_j)^i \cdot \text{ct}_i[1]$  for all  $j$ . Thus, any two sets of accepted pre-decryption keys for the same batch of ciphertexts are equal, they give the same set of messages because decryption is a deterministic algorithm.

Accurate blaming and consistency together imply the robustness of the weighted threshold construction.  $\square$

### 3.1 Security Assumption and CCA Security

We use the following weighted analog of the decisional  $n$ -power Diffie–Hellman assumption used by BTX [4, Assumption 4.1].

**Assumption 2** (Weighted Decisional  $d$ -Power Diffie–Hellman WDdP-DH). *Given parameters  $(N, W, (w_j)_{j \in [N]}, (\Omega_j)_{j \in [N]}, t, \ell)$ , define the two-stage experiment  $\text{WDdP}_{\mathcal{B}}^b(1^\lambda)$ , for  $b \in \{0, 1\}$  and distinguisher  $\mathcal{B} = (\mathcal{B}_1, \mathcal{B}_2)$ , as follows. The challenger samples the description of the bilinear groups,  $z, a_1, \dots, a_t \leftarrow \mathbb{F}_p$ , and  $q_j \leftarrow \mathbb{F}_p^*$  for every  $j \in [N]$ . It sets*

$$Z(X) = z + \sum_{s=1}^t a_s X^s, \quad z_\omega = Z(\omega),$$

and computes

$$\begin{aligned} \text{ek} &= [z]_T, \\ \text{vk}_{j,i} &= [q_j^i]_2 & (j \in [N], i \in [\ell]), \\ \text{dk}_{\omega,d} &= [z_\omega q_{\text{own}(\omega)}^d]_2 & (\omega \in \Omega_{[N]}, d \in \{-\ell, \dots, -1, 1, \dots, \ell-1\}), \\ H_{j,i} &= [q_j^i]_1 & (j \in [N], i \in [\ell]). \end{aligned}$$

Let  $\text{pub} = ((\Omega_j)_{j \in [N]}, \text{ek}, (\text{vk}_{j,i})_{j,i}, (\text{dk}_{\omega,d})_{\omega,d})$ . In the first stage,

$$(\mathcal{C} \subseteq [N], \text{st}) \leftarrow \mathcal{B}_1(\text{pub}, (H_{j,i})_{j,i}).$$

The experiment returns 0 if  $\text{wt}(\mathcal{C}) > t$ . Otherwise, the challenger samples  $r, u \leftarrow \mathbb{F}_p$ , sets

$$X = [r]_1, \quad Y_0 = [rz]_T, \quad Y_1 = [u]_T,$$

and returns the output of

$$\mathcal{B}_2(\text{st}, (q_j)_{j \in \mathcal{C}}, X, Y_b).$$

Define

$$\begin{aligned} \text{Adv}_{\mathcal{B}}^{\text{WDdP-DH}}(\lambda) &= |\Pr[\text{WDdP}_{\mathcal{B}}^0(1^\lambda) = 1] \\ &\quad - \Pr[\text{WDdP}_{\mathcal{B}}^1(1^\lambda) = 1]|. \end{aligned}$$

The WDdP-DH assumption states that this advantage is negligible for every PPT distinguisher  $\mathcal{B}$  and every polynomially bounded choice of valid parameters.

The elements  $(H_{j,i})_{j,i}$  are used by the simulation and are given to the distinguishing adversary, but are not public parameters of Construction 1. The assumption is falsifiable, as both distributions in the experiment are efficiently samplable.

**Theorem 3 (Static WB-IND-CCA Security).** *Suppose Assumption 2 holds and  $\Pi_{\text{DL}}$  is complete, zero knowledge, and straightline, simulation extractable for  $R_{\text{DL}}$ . Then, Construction 1 is WB-IND-CCA secure.*

[Sasha: Does straightline need to be defined/assumed as a property of Pi DL?][Nusret: Yes, no need to define it I guess, it is good to learn what it is. We need it here so that we do not have to rewind the adversary.]

*Proof.* Let  $\mathcal{A}$  be the WB-IND-CCA adversary (Definition 3). We consider the following games, and write  $S_k$  for the event that  $\mathcal{A}$  outputs the challenge bit in Game  $k$ .

**Game 0.** This is the real WB-IND-CCA game. Hence

$$\left| \Pr[S_0] - \frac{1}{2} \right| = \text{Adv}_{\text{WB-TX}, \mathcal{A}}^{\text{WB-IND-CCA}}(\lambda).$$

**Game 1.** At setup, we replace the NIZK CRS by

$$(\text{crs}, \text{td}_{\text{sim}}, \text{td}_{\text{ext}}) \leftarrow \Pi_{\text{DL}}.\text{SimSetup}(1^\lambda),$$

and generate the proof within the challenge ciphertext using  $\Pi_{\text{DL}}.\text{SimProve}$ . The zero-knowledge property implies that there is a negligible function  $\epsilon_{\text{zk}}$  such that

$$|\Pr[S_0] - \Pr[S_1]| \leq \epsilon_{\text{zk}}.$$

**Game 2.** We change the implementation of the pre-decryption oracle. Given a query  $(\text{ct}_i)_{i \in I}$ , where  $\text{ct}_i = (\text{ct}_i[1], \text{ct}_i[2], \pi_i)$  and  $I \subseteq [\ell]$ , the oracle first performs the prescribed verification and returns  $\perp$  if a proof is invalid. It then computes  $r_i \leftarrow \Pi_{\text{DL}}.\text{Ext}(\text{crs}, \text{td}_{\text{ext}}, (\text{ct}_i[1], \text{ct}_i[2]), \pi_i)$  for every accepting ciphertext, and returns

$$\widehat{\text{sbk}}_j = \sum_{i \in I} r_i H_{j,i}$$

for every  $j \notin \mathcal{C}$ . Observe that if  $\text{ct}_i[1] = [r_i]_1$  for all queried ciphertexts, then

$$\widehat{\text{sbk}}_j = \sum_{i \in I} r_i H_{j,i} = \sum_{i \in I} [r_i q_j^i]_1 = \sum_{i \in I} q_j^i \text{ct}_i[1]$$

is exactly the output of  $\text{PreDec}$ .

Before the challenge, the simulator has returned no statement-proof pair. After the challenge, the only such pair is the challenge pair  $((\text{ct}^*[1], \text{ct}^*[2]), \pi^*)$ . The WB-IND-CCA game rejects a query containing the exact challenge ciphertext. Hence, every pair on which the extractor is invoked is outside the simulator's output set, including the pairs which use  $(\text{ct}^*[1], \text{ct}^*[2])$  as the statement along with a fresh proof. Then, by the simulation-extractability of the NIZK and a union bound over all queried ciphertexts, the probability that  $\text{ct}_i[1] \neq [r_i]_1$  for an extracted  $r_i \leftarrow \Pi_{\text{DL}}.\text{Ext}(\text{crs}, \text{td}_{\text{ext}}, (\text{ct}_i[1], \text{ct}_i[2]), \pi_i)$  is bounded by a negligible function  $\epsilon_{\text{ext}}$ . Therefore, it follows that

$$|\Pr[S_1] - \Pr[S_2]| \leq \epsilon_{\text{ext}}.$$

**Game 3.** Replace the pad  $[r^*z]_T$  in the challenge ciphertext by an independent uniform element of  $\mathbb{G}_T$ . We reduce this change to Assumption 2 with the distinguishing adversary  $\mathcal{B}$ . Given  $(\text{pub}, (H_{j,i})_{j,i})$ , the distinguisher  $\mathcal{B}$  generates the

simulated NIZK CRS and passes  $\text{pub}$  and the CRS to  $\mathcal{A}$ . It forwards the static corruption set chosen by  $\mathcal{A}$  and gives the returned  $(q_j)_{j \in \mathcal{C}}$  to  $\mathcal{A}$ . It answers pre-decryption queries as in Game 2. Upon receiving  $m_0, m_1$ , it samples  $\mu \leftarrow \{0, 1\}$  and uses the received challenge  $(X, Y_b)$  to form

$$\text{ct}^* = (X, m_\mu + Y_b, \Pi_{\text{DL}}.\text{SimProve}(\text{crs}, \text{td}_{\text{sim}}, (X, m_\mu + Y_b))).$$

Finally,  $\mathcal{B}$  returns 1 if and only if  $\mathcal{A}$  outputs  $\mu$ .

Now, if  $b = 0$ , Game 3 is identical to Game 2. Consequently,

$$|\Pr[S_2] - \Pr[S_3]| \leq \text{Adv}_{\mathcal{B}}^{\text{WDdP-DH}}(\lambda).$$

In Game 3, the second component of the challenge ciphertext is uniform and independent of  $\mu$ , so  $\Pr[S_3] = 1/2$ . Combining the games gives

$$\text{Adv}_{\text{WB-TX}, \mathcal{A}}^{\text{WB-IND-CCA}}(\lambda) \leq \text{Adv}_{\mathcal{B}}^{\text{WDdP-DH}}(\lambda) + \epsilon_{\text{zk}} + \epsilon_{\text{ext}},$$

which is negligible.  $\square$

**Lemma 1 (Bilinear generic group hardness).** *Assumption 2 holds in the bilinear generic group model except with probability  $O(Q^2(2\ell + t)/p)$ , where  $Q$  denotes the number of generic group operations.*

*Proof.* Fix the set  $\mathcal{C} \subseteq [N]$  returned in the first stage and reveal  $(q_j)_{j \in \mathcal{C}}$  to the adversary. In the symbolic game, we treat

$$r, z, a_1, \dots, a_t, (q_j)_{j \notin \mathcal{C}}$$

as algebraically independent indeterminates. Then, producing  $rz$  requires  $z$  to lie in the linear span of the Laurent polynomials

$$1, \quad q_j^i, \quad Z(\omega)q_{\text{own}(\omega)}^d \quad (w \in \Omega_{[N]}, i \in [\ell], d \in \{-\ell, \dots, -1, 1, \dots, \ell - 1\}).$$

Now, consider a party  $j \notin \mathcal{C}$ . Since every  $d$  is nonzero, each term  $Z(\omega)q_j^d$ ,  $\omega \in \Omega_j$ , contains a non-zero power of the independent indeterminate  $q_j$ . Comparing the coefficients shows that any combination of these terms which is independent of  $q_j$  contributes zero. The same observation applies to the terms  $q_j^i$ . On the other hand, the terms belonging to the corrupted parties reduce to the span of  $(Z(\omega))_{\omega \in \Omega_{\mathcal{C}}}$ . Thus, producing  $rz$  requires  $z$  to in fact lie in the linear span of  $(Z(\omega))_{\omega \in \Omega_{\mathcal{C}}}$ , which we show to be false by contradiction.

Suppose  $z = c_0 + \sum_{\omega \in \Omega_{\mathcal{C}}} c_\omega Z(\omega)$  for coefficients in  $\mathbb{F}_p$ . Comparing the coefficients of  $a_s$ , for  $s = 1, \dots, t$ , gives

$$\sum_{\omega \in \Omega_{\mathcal{C}}} c_\omega \omega^s = 0.$$

Since the points are distinct and nonzero and  $|\Omega_{\mathcal{C}}| \leq t$ , the corresponding Vandermonde matrix has full column rank, which implies  $c_\omega = 0$  for  $\omega \in \Omega_{\mathcal{C}}$ . Comparing the coefficient of  $z$  then gives the desired contradiction.

Finally, the adversary can distinguish the symbolic and real games only if two distinct formal expressions in the symbolic game receive the same value in real game. Let

$$D := \prod_{j \notin \mathcal{C}} q_j^\ell \quad \text{and} \quad M := (N - |\mathcal{C}|)\ell$$

so that multiplication by  $D$  clears every negative power of an honest party's  $q_j$ . The exponents represented in  $\mathbb{G}_1$  have degree at most  $\ell$ , while the cleared numerators of the exponents represented in  $\mathbb{G}_2$  have degree at most  $M + \ell$ . Consequently, a pairing yields degree at most  $M + 2\ell \leq (N + 2)\ell$ . Hence, by the Schwartz–Zippel lemma and a union bound over at most  $O(Q^2)$  pairs of expressions, after clearing the denominators of the Laurent polynomials, the adversary can distinguish the symbolic and real games with probability at most  $O(Q^2(N + 2)\ell/p)$ , which gives the stated bound.  $\square$

### 3.2 Cost analysis for Construction 1

In Table 1, we give the storage/communication costs for the Weighted BTX scheme. In Table 2, we give computation costs for the Weighted BTX scheme.

### 3.3 DKG

We describe a distributed key-generation protocol for Construction 1 in the synchronous network model. The protocol treats Byzantine broadcast as a black-box primitive and has two *logical* rounds, one for generating the  $q_j$ -powers and one for generating the sharing polynomial and the public decryption material. itself require several synchronous communication rounds.

*Informal sketch.* The DKG protocol has two steps. In the first step, every party  $P_j$  samples its secret key  $q_j \leftarrow \mathbb{F}_p^*$ . It broadcasts  $\overline{Q}_j := [q_j]_1$  and  $Q_{j,d} := [q_j^d]_2$  for  $d \in \{-\ell, \dots, -1, 1, \dots, \ell\}$ . The positive powers form the party's verification key, while the remaining powers are needed to construct the public decryption material. To prove that all these elements contain powers of the same scalar,  $P_j$  gives an extractable discrete-log proof of knowledge of  $q_j$  relative to  $\overline{Q}_j$  and signs the complete transcript. Taking  $Q_{j,0} = [1]_2$ , a recipient verifies the chain of pairing equations

$$e(\overline{Q}_j, Q_{j,d}) = e([1]_1, Q_{j,d+1}) \quad \text{for every } d \in \{-\ell, \dots, \ell - 1\}.$$

The proof of knowledge binds  $\overline{Q}_j$  to a known scalar, and the pairing chain then forces every  $Q_{j,d}$  to be the corresponding power of that same scalar, including the negative powers.

In the second step, each dealer  $P_i$  samples an independent uniformly random polynomial  $f_i(X) \in \mathbb{F}_p[X]$  of degree at most  $t$ . It sets  $z^{(i)} := f_i(0)$  and  $z_\omega^{(i)} := f_i(\omega)$  for every  $\omega \in \Omega_J$ , and computes

$$Z_{i,0} := [z^{(i)}]_1, \quad Z_{i,T} := [z^{(i)}]_T, \quad Z_{i,\omega} := [z_\omega^{(i)}]_1,$$

Table 1: Storage and communication costs for Construction 1.  $N$  is the number of real parties,  $W$  is the total weight,  $\tau = |T|$  is the number of parties in the accepted set, and  $\ell$  is the batch size. We exclude costs related to the NIZK.

| Quantity                                                              | Size                                                    |
|-----------------------------------------------------------------------|---------------------------------------------------------|
| Encryption key $\mathbf{ek}$                                          | $ \mathbb{G}_T $                                        |
| Party secret key $\mathbf{sk}_j$                                      | $ \mathbb{F}_p $                                        |
| Party verification key $\mathbf{vk}_j$                                | $\ell \mathbb{G}_2 $                                    |
| All parties' verification keys                                        | $N\ell \mathbb{G}_2 $                                   |
| Core decryption material $(D_{w,d})$                                  | $(2\ell - 1)W \mathbb{G}_2 $                            |
| Public decryption key $\mathbf{dk}$ , including all verification keys | $((2\ell - 1)W + N\ell) \mathbb{G}_2 $                  |
| Non-NIZK CRS / total public parameters $(\mathbf{ek}, \mathbf{dk})$   | $((2\ell - 1)W + N\ell) \mathbb{G}_2  +  \mathbb{G}_T $ |
| Ciphertext, excluding $\pi_{\text{DL}}$                               | $ \mathbb{G}_1  +  \mathbb{G}_T $                       |
| Padded batch of ciphertexts, excluding proofs                         | $\ell \mathbb{G}_1  + \ell \mathbb{G}_T $               |
| One party's pre-decryption $\sigma_j$                                 | $ \mathbb{G}_1 $                                        |
| Pre-decryptions from an accepted set $T$                              | $\tau \mathbb{G}_1 $                                    |
| Decrypted message batch                                               | $\ell \mathbb{G}_T $                                    |

Table 2: Computation costs for Construction 1. For  $a \in \{1, 2, T\}$ ,  $\mathbf{SM}_a$  is a scalar multiplication in  $\mathbb{G}_a$  and  $\mathbf{MSM}_a(q)$  is a multi-scalar multiplication of size  $q$  in  $\mathbb{G}_a$ . Group additions are not counted.  $\mathbf{P}$  denotes a pairing,  $\mathbf{MP}(q)$  denotes a multi-pairing of size  $q$ , and  $\mathbf{FFT}_a(q)/\mathbf{IFFT}_a(q)$  denote size  $q$  FFT's in  $\mathbb{G}_a$ . We set  $m = 2^{\lceil \log_2(2\ell) \rceil}$ .

| Operation                                | Cost                                                                                                                                                                        |
|------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Enc, one ciphertext                      | $1\mathbf{SM}_1 + 1\mathbf{SM}_T$                                                                                                                                           |
| PreDec $_j$ , one padded batch           | $1\mathbf{MSM}_1(\ell)$                                                                                                                                                     |
| Verify one pre-decryption $\sigma_j$     | $1\mathbf{MP}(\ell + 1)$                                                                                                                                                    |
| Dec, naive cross-term evaluation         | $\ell\mathbf{MP}(\tau + \ell - 1) + \sum_{j \in T} \ell\mathbf{MSM}_2(w_j) + (\ell - 1)\mathbf{MSM}_2(W_T)$                                                                 |
| Dec, FFT-based middle-product evaluation | $\ell\mathbf{MP}(\tau) + m\mathbf{P} + \mathbf{FFT}_1(m) + \sum_{j \in T} \ell\mathbf{MSM}_2(w_j) + (\ell - 1)\mathbf{MSM}_2(W_T) + \mathbf{FFT}_2(m) + \mathbf{IFFT}_T(m)$ |

together with

$$D_{i,\omega,d} := [z_{\omega}^{(i)} q_{\text{own}(\omega)}^d]_2 \quad \text{for every } \omega \in \Omega_J \text{ and } d \in \{-\ell, \dots, -1, 1, \dots, \ell - 1\}.$$

To prevent rogue-key attacks, we require each party to also provide an extractable discrete-log proof of knowledge of  $z^{(i)}$  relative to  $Z_{i,0}$ . It then broadcasts the resulting transcript to all parties.

A recipient first verifies the proof of knowledge. It then checks  $e(Z_{i,0}, [1]_2) = Z_{i,T}$ , applies the SCRAPE low-degree test to  $(Z_{i,x})_{x \in \{0\} \cup \Omega_J}$ , and verifies, for every relevant  $\omega$  and  $d$ , that

$$e(Z_{i,\omega}, Q_{\text{own}(\omega),d}) = e([1]_1, D_{i,\omega,d}).$$

Thus, the share commitments lie on one degree  $\leq t$  polynomial whose constant term is bound to the proof, the target-group commitment is consistent with that constant, and every decryption-key element uses the correct share and  $q$ -power.

Finally, each party locally adds up the corresponding components of all the valid transcripts. For instance,  $Z = \Sigma_{i \in \mathcal{J}} [z^{(i)}]_1$ , where  $\mathcal{J}$  is the set of parties whose Round 2 transcripts were valid.

For the security proofs, We assume that both proofs of knowledge admit straight-line extraction and bind the session, logical round, dealer identity, and complete public statement.

### Byzantine broadcast and DKG definition

**Definition 10 (Byzantine broadcast).** *A Byzantine-broadcast instance  $\text{BB}[\text{sid}, r, j]$  has a designated broadcaster  $P_j$ , an instance identifier consisting of a session identifier  $\text{sid}$  and a logical-round label  $r$ , and an initial input held by  $P_j$ . It satisfies:*

- **Agreement.** *If two honest parties output  $M$  and  $M'$  in the same instance, then  $M = M'$ .*
- **Validity.** *If  $P_j$  is honest and has input  $M$ , then every honest party outputs  $M$ .*
- **Termination.** *Every honest party outputs a message.*

*In the synchronous setting, the implementation has a publicly known finite round bound. Hence every honest party has produced exactly one output—which may be a distinguished default value  $\perp$ —from every concurrently invoked instance by a common deadline.*

Let  $P_1, \dots, P_N$  have respective integer weights  $(w_j)_{j \in [N]}$ , let  $W := \sum_{j \in [N]} w_j$ , and let the adversary  $\mathcal{A}$  statically corrupt parties of total weight at most  $t$ . We assume  $W \geq 2t + 1$ .

We now formally define weighted Distributed Key Generation. We adapt the definition of [21] to the public material required by our construction.

**Definition 11 (Weighted BTX DKG correctness).** *A terminating DKG execution outputs a qualified owner set  $J$ , an auxiliary constant-term commitment  $Z_0 \in \mathbb{G}_1$ , auxiliary evaluation commitments  $(Z_\omega)_{\omega \in \Omega_J}$ , an encryption key  $\text{ek} \in \mathbb{G}_T$ , verification keys  $(\text{vk}_j)_{j \in J}$ , and public decryption material  $\text{dk}$ . Write  $Z_0 = [z]_1$  and  $Z_\omega = [z_\omega]_1$  for their unique exponents. The DKG satisfies the following properties, except with probability  $\text{negl}(\lambda)$ :*

- (C1) *Every subset  $S \subseteq \Omega_J$  of  $t + 1$  distinct evaluation points defines the same unique master value:*

$$\sum_{\omega \in S} \lambda_{S, \omega}(0) Z_\omega = Z_0 = [z]_1,$$

*where  $\lambda_{S, \omega}(0)$  is the Lagrange coefficient for interpolation at zero.*

- (C2) All honest parties output the same encryption key  $\mathbf{ek} = [z]_T$ , where  $z$  is the exponent of the unique value guaranteed by C1; equivalently,  $\mathbf{ek} = e(Z_0, [1]_2)$ .
- (C3) All honest parties agree on  $J$  and output the same verification keys and public decryption material. For a single nonzero scalar  $q_j$  associated with each  $j \in J$ , these values have the form

$$\mathbf{vk}_{j,k} = [q_j^k]_2 \quad (j \in J, k \in [\ell]), \quad \mathbf{dk}_{\omega,d} = [z_\omega q_{\text{own}(\omega)}^d]_2$$

for every  $\omega \in \Omega_J$  and  $d \in \{-\ell, \dots, -1, 1, \dots, \ell - 1\}$ .

As in [21], C1–C3 are correctness and agreement properties; they do not assert that  $z$  is uniformly distributed. We will instead our scheme to be secure in combination with the DKG protocol.

**Protocol** The public input consists of  $\mathbf{pp}$ , the weights, the threshold  $t$ , the batch bound  $\ell$ , the evaluation domains, and a fresh session identifier  $\mathbf{sid}$ . Let

$$\mathcal{E}_\ell := \{-\ell, \dots, -1, 1, \dots, \ell\}, \quad \mathcal{D}_\ell := \{-\ell, \dots, -1, 1, \dots, \ell - 1\}.$$

All transcript encodings and validity checks are deterministic. Every verifier first checks canonical encodings and group and subgroup membership. The protocol uses a straight-line extractable proof of knowledge  $\Pi_{\text{DL}}$  for the relation  $\{(U; u) : U = [u]_1\}$ .

*SCRAPE low-degree test.* For completeness, let  $X \subseteq \mathbb{F}_p$  be a set of  $m$  distinct points and let  $(V_x)_{x \in X} \in \mathbb{G}_1^m$ . After the vector is fixed, SCRAPE samples a uniform polynomial  $r(X)$  of degree at most  $m - t - 2$  and defines

$$c_x := \frac{r(x)}{\prod_{y \in X \setminus \{x\}} (x - y)} \quad (x \in X).$$

It accepts if

$$\sum_{x \in X} c_x V_x = [0]_1. \quad (2)$$

The test has perfect completeness for evaluations of a polynomial of degree at most  $t$  and soundness error at most  $1/p$  for a fixed invalid vector. In the noninteractive protocol below, the challenge is derived after the entire Byzantine-broadcast output is fixed, using a domain-separated Fiat–Shamir hash of the session, round, dealer, point set, and complete commitment vector. We assume the resulting noninteractive test is sound; because its challenge derivation is deterministic, all honest parties make the same decision.

**Contribution of  $P_j$ :**

- ▷ Sample  $q_j \leftarrow \mathbb{F}_p^*$  and compute

$$\overline{Q}_j := [q_j]_1, \quad Q_{j,0} := [1]_2, \quad Q_{j,d} := [q_j^d]_2 \quad (d \in \mathcal{E}_\ell).$$

- ▷ Let the proof statement be

$$S_j^q := (\text{sid}, Q, j, \overline{Q}_j, (Q_{j,d})_{d \in \mathcal{E}_\ell})$$

and compute

$$\pi_j^q \leftarrow \Pi_{\text{DL}}.\text{Prove}(S_j^q; q_j).$$

- ▷ Invoke  $\text{BB}[\text{sid}, Q, j]$  with input

$$M_j^q := (S_j^q, \pi_j^q).$$

**Validation and qualification:**

- ▷ Wait until all  $N$  Round-1 Byzantine-broadcast instances have terminated. Let  $\widetilde{M}_j^q$  be the output of the instance whose designated broadcaster is  $P_j$ .
- ▷ Declare  $\widetilde{M}_j^q$  valid if it has the prescribed context and format,  $\Pi_{\text{DL}}.\text{Verify}$  accepts  $\pi_j^q$ , and, taking  $Q_{j,0} = [1]_2$ , the following holds for every  $d \in \{-\ell, \dots, \ell-1\}$ :

$$e(\overline{Q}_j, Q_{j,d}) = e([1]_1, Q_{j,d+1}). \quad (3)$$

Here  $Q_{j,d}$  or  $Q_{j,d+1}$  denotes  $Q_{j,0} = [1]_2$  when the corresponding index is zero.

- ▷ Set

$$J := \{j \in [N] : \widetilde{M}_j^q \text{ is valid}\}, \quad \Omega_J := \bigcup_{j \in J} \Omega_j.$$

Abort if  $|\Omega_J| \leq t$ . Otherwise, freeze  $J$ , the accepted Round-1 transcripts, and  $\Omega_J$  before starting Round 2.

**Construction 2:** Synchronous DKG, Round 1: generation of the  $q$ -powers.

Let  $\text{ctx}_J$  denote the Round-2 context consisting of the original public input,  $J$ , and the canonical sender-indexed list of accepted Round-1 transcripts. Binding the Round-2 proof and SCRAPE challenge to this context prevents replay across sessions or across different qualified-owner sets.

**Contribution of  $P_i$ :**

- ▷ Sample a uniformly random polynomial  $f_i(X) \in \mathbb{F}_p[X]$  of degree at most  $t$ . Define  $z^{(i)} := f_i(0)$  and  $z_\omega^{(i)} := f_i(\omega)$  for every  $\omega \in \Omega_J$ .
- ▷ Compute

$$Z_{i,0} := [z^{(i)}]_1, \quad Z_{i,T} := [z^{(i)}]_T, \quad Z_{i,\omega} := [z_\omega^{(i)}]_1 \quad (\omega \in \Omega_J),$$

and, for every  $\omega \in \Omega_J$  and  $d \in \mathcal{D}_\ell$ , compute

$$D_{i,\omega,d} := z_\omega^{(i)} Q_{\text{own}(\omega),d} = [z_\omega^{(i)} q_{\text{own}(\omega)}^d]_2.$$

- ▷ Let the proof statement be

$$S_i^z := (\text{ctx}_J, Z, i, Z_{i,0}, Z_{i,T}, (Z_{i,\omega})_{\omega \in \Omega_J}, (D_{i,\omega,d})_{\substack{\omega \in \Omega_J \\ d \in \mathcal{D}_\ell}})$$

and compute

$$\pi_i^z \leftarrow \Pi_{\text{DL}}.\text{Prove}(S_i^z; z^{(i)}).$$

Invoke  $\text{BB}[\text{sid}, Z, i]$  with input

$$M_i^z := (S_i^z, \pi_i^z).$$

**Construction 3:** Synchronous DKG, Round 2: polynomial contributions and key derivation.

**Validation:**

- ▷ Wait until all  $N$  Round-2 Byzantine-broadcast instances have terminated. Let  $\widetilde{M}_i^z$  be the output of the instance whose designated broadcaster is  $P_i$ .
- ▷ Declare  $\widetilde{M}_i^z$  valid only if it has the prescribed context and format and all the following checks pass:
  1. verify  $\pi_i^z$  using  $\Pi_{\text{DL}}.\text{Verify}$ ;
  2. check consistency of the two constant-term commitments,

$$e(Z_{i,0}, [1]_2) = Z_{i,T}; \quad (4)$$

3. run the SCRAPE test on  $(Z_{i,x})_{x \in \{0\} \cup \Omega_J}$  with degree bound  $t$  and a challenge bound to  $(\text{ctx}_J, i)$ ;
4. for every  $\omega \in \Omega_J$  and  $d \in \mathcal{D}_\ell$ , check

$$e(Z_{i,\omega}, Q_{\text{own}(\omega),d}) = e([1]_1, D_{i,\omega,d}). \quad (5)$$

- ▷ Define the valid Round-2 dealer set

$$I := \{i \in [N] : \widetilde{M}_i^z \text{ is valid}\}.$$

Abort if  $\text{wt}(I) \leq t$ .

**Aggregation and output:**

- ▷ Compute, componentwise,

$$\begin{aligned} Z_0 &:= \sum_{i \in I} Z_{i,0}, & Z_T &:= \sum_{i \in I} Z_{i,T}, \\ Z_\omega &:= \sum_{i \in I} Z_{i,\omega}, & D_{\omega,d} &:= \sum_{i \in I} D_{i,\omega,d} \end{aligned}$$

for every  $\omega \in \Omega_J$  and  $d \in \mathcal{D}_\ell$ .

- ▷ Output

$$\text{ek} := Z_T, \quad \text{vk}_j := (Q_{j,k})_{k \in [\ell]} \quad (j \in J), \quad \text{dk} := (D_{\omega,d})_{\substack{\omega \in \Omega_J \\ d \in \mathcal{D}_\ell}}.$$

The auxiliary commitments  $Z_0$  and  $(Z_\omega)_{\omega \in \Omega_J}$  may be retained as part of the public DKG transcript. Every honest  $P_j \in J$  retains  $q_j$  as its secret key.

**Construction 4:** Synchronous DKG, Round 2: validation and key derivation.

**Proof of C1–C3**

**Theorem 4 (Correctness of the synchronous DKG).** *Suppose the bilinear groups have prime order and a nondegenerate pairing,  $\Pi_{\text{DL}}$  is sound and admits straight-line extraction, and the noninteractive SCRAPE test is sound. Suppose also that every sender-indexed invocation of Byzantine broadcast satisfies Definition 10. Then, against every PPT adversary  $\mathcal{A}$  corrupting parties of total weight at most  $t$ , the protocol of Constructions 2, 3, and 4 terminates and satisfies C1–C3 of Definition 11, except with probability  $\text{negl}(\lambda)$ .*

*Proof.* Condition on the event that no proof-of-knowledge soundness or extraction failure and no SCRAPE soundness failure occurs. By assumption, the complement of this event has probability  $\text{negl}(\lambda)$ .

*Common Round-1 state and well-formed powers.* For each sender  $P_j$ , Byzantine-broadcast Agreement and Termination imply that all honest parties obtain the same output  $\widetilde{M}_j^q$  by the Round-1 deadline. Thus, they obtain the same sender-indexed output vector. Since transcript validation is deterministic, they derive the same set  $J$ , the same accepted transcripts, and the same  $\Omega_J$ . Byzantine-broadcast Validity and completeness of the checks imply that every honest party belongs to  $J$ . Therefore

$$|\Omega_J| = \text{wt}(J) \geq \text{wt}(\mathcal{H}) \geq W - t \geq t + 1,$$

where  $\mathcal{H}$  is the set of honest parties; hence an honest execution does not abort at the qualification check.

For every  $j \in J$ , straight-line extraction from  $\pi_j^q$  yields a scalar  $q_j$  such that  $\overline{Q}_j = [q_j]_1$ . Write  $Q_{j,d} = [a_{j,d}]_2$  and set  $a_{j,0} = 1$ . By equation (3) and nondegeneracy of the pairing,

$$q_j a_{j,d} = a_{j,d+1} \quad (-\ell \leq d \leq \ell - 1). \quad (6)$$

For  $d = -1$ , this gives  $q_j a_{j,-1} = 1$ , so  $q_j \neq 0$ . Induction forward and backward from  $a_{j,0} = 1$  now gives

$$Q_{j,d} = [q_j^d]_2 \quad (d \in \mathcal{E}_\ell). \quad (7)$$

*Common Round-2 state and aggregate polynomial.* Round 2 begins only after the common Round-1 state is frozen. Applying the same Agreement, Termination, and deterministic-validation argument to the  $N$  Round-2 instances shows that all honest parties derive the same set  $I$  and the same accepted transcripts. Validity and completeness again imply that every honest party belongs to  $I$ . Consequently,  $\text{wt}(I) \geq \text{wt}(\mathcal{H}) \geq W - t \geq t + 1$ , so the Round-2 qualification check does not cause an honest execution to abort.

For each  $i \in I$ , extraction from  $\pi_i^z$  yields  $z^{(i)}$  such that  $Z_{i,0} = [z^{(i)}]_1$ . SCRAPE soundness gives a polynomial  $f_i(X) \in \mathbb{F}_p[X]$  of degree at most  $t$  such that

$$Z_{i,0} = [f_i(0)]_1, \quad Z_{i,\omega} = [f_i(\omega)]_1 \quad (\omega \in \Omega_J). \quad (8)$$

Uniqueness of exponents in the prime-order source group implies  $f_i(0) = z^{(i)}$ . Equation (4) then gives  $Z_{i,T} = [z^{(i)}]_T$ . Moreover, writing  $D_{i,\omega,d} = [b_{i,\omega,d}]_2$ , equations (5) and (7), together with nondegeneracy, imply

$$b_{i,\omega,d} = f_i(\omega)q_{\text{own}(\omega)}^d. \quad (9)$$

Define

$$F(X) := \sum_{i \in I} f_i(X), \quad z := F(0) = \sum_{i \in I} z^{(i)}, \quad z_\omega := F(\omega).$$

Since every  $f_i$  has degree at most  $t$ , so does  $F$ . Componentwise aggregation and equations (8) and (9) give

$$Z_0 = [z]_1, \quad Z_\omega = [z_\omega]_1, \quad Z_T = [z]_T, \quad D_{\omega,d} = [z_\omega q_{\text{own}(\omega)}^d]_2. \quad (10)$$

C1. Let  $S \subseteq \Omega_J$  be any set of  $t+1$  distinct points. Since  $\deg(F) \leq t$ , Lagrange interpolation and equation (10) yield

$$\begin{aligned} \sum_{\omega \in S} \lambda_{S,\omega}(0) Z_\omega &= \left[ \sum_{\omega \in S} \lambda_{S,\omega}(0) F(\omega) \right]_1 \\ &= [F(0)]_1 = Z_0 = [z]_1. \end{aligned}$$

The result is independent of  $S$ , and  $t+1$  distinct evaluations uniquely determine a degree-at-most- $t$  polynomial. Hence the master value is unique, which proves C1.

C2. All honest parties use the same set  $I$  and the same sender-indexed Round-2 transcripts, so they compute the same  $Z_T$ . By equation (10),

$$\text{ek} = Z_T = [z]_T = e([z]_1, [1]_2) = e(Z_0, [1]_2),$$

where  $z$  is precisely the unique value established by C1. This proves C2.

C3. The common Round-1 vector gives all honest parties the same  $J$  and the same verification keys. Equation (7) shows that  $\text{vk}_{j,k} = Q_{j,k} = [q_j^k]_2$  for every  $j \in J$  and  $k \in [\ell]$ . The common Round-2 vector and the deterministic set  $I$  give all honest parties the same componentwise aggregate. Finally, equation (10) shows that every element of this aggregate has the required form  $\text{dk}_{\omega,d} = [z_\omega q_{\text{own}(\omega)}^d]_2$ . Thus all honest parties agree on, and output, the same well-formed verification keys and public decryption material. This proves C3.  $\square$

*Remark 2 (Scope of the theorem).* The theorem proves C1–C3, exactly the correctness scope of the weighted-VUF DKG definition. Standard Byzantine broadcast does not provide simultaneous input commitment: a rushing corrupt Round-2 dealer may choose a known valid contribution, or choose whether to submit a valid contribution, after seeing honest public commitments. The proofs of knowledge prevent unknown-exponent cancellation but do not make the final

exponent statistically uniform. Consequently, using this DKG as a replacement for the trusted setup of Construction 1 additionally requires either a direct security proof for the resulting (possibly biased) key distribution, or a hide–fix–reveal DKG with recovery of every contribution fixed before it is revealed.

*Remark 3 (Broadcast realization).* The protocol uses Byzantine broadcast as an assumed primitive. Any concrete realization must tolerate the actual number of Byzantine identities allowed by the weighted corruption model and must provide instance separation and a known synchronous termination bound. No application-level signature is needed in the protocol above, although a concrete authenticated Byzantine- broadcast implementation may use signatures internally.

## 4 Partial Fraction Based Weighted BTE

Our second construction, presented in Construction 5 is a weighted batch threshold encryption scheme based on ideas from [12,10,3]. For  $a \in \mathbb{F}_p$ , define  $p_a(X) := 1/(X + a)$ . We fix distinct public  $\alpha_1, \dots, \alpha_\ell \in \mathbb{F}_p \setminus \{0, -1\}$ . Decryption uses the partial fraction identity

$$g_{j,k}g_{j,i} = \frac{g_{j,i} - g_{j,k}}{\alpha_k - \alpha_i} \quad (k \neq i). \quad (11)$$

We can again accelerate the decryption algorithm using multi-pairings and fast Fourier transform (FFT). The first terms in  $\sum_{j \in T'} L_{j,i}$  can be evaluated as one multi-pairing of arity  $|T'|$  for each  $i \in [\ell]$ . For the remaining terms,

$$\sum_{j \in T'} \sum_{\substack{k \in [\ell] \\ k \neq i}} e \left( \frac{\text{ct}_k[1]}{\alpha_k - \alpha_i}, \sum_{w \in \Omega_j} \lambda_{T',w} \text{dk}_{w,k}[1] \right) = \sum_{\substack{k \in [\ell] \\ k \neq i}} \frac{E_k}{\alpha_k - \alpha_i},$$

where

$$E_k := e \left( \text{ct}_k[1], \sum_{w \in \Omega_{T'}} \lambda_{T',w} \text{dk}_{w,k}[1] \right).$$

Each  $E_k$  is independent of  $i$  and is therefore computed once and reused for every output ciphertext. This requires  $\ell$  ordinary pairings. Moreover, the two remaining pairings with left input  $\text{ct}_i[1]$  can be combined as

$$(\alpha_i + 1)e(\text{ct}_i[1], D_i) - e(\text{ct}_i[1], \text{dk}[0]) = e(\text{ct}_i[1], (\alpha_i + 1)D_i - \text{dk}[0]),$$

where  $D_i := \sum_{w \in \Omega_{T'}} \lambda_{T',w} \text{dk}_{w,i}[2]$ . Consequently, decryption requires  $2\ell$  pairings in addition to the  $\ell$  multi-pairings of size  $|T'|$ , implying  $|T'| + 2$  pairing inputs per ciphertext.

The remaining quadratic work in decryption consists of Cauchy sums of the form

$$\mathcal{C}_i(X) := \sum_{\substack{k \in [\ell] \\ k \neq i}} \frac{X_k}{\alpha_k - \alpha_i},$$

where  $(X_k)_k$  is either  $(\text{ct}_k[1])_k \subseteq \mathbb{G}_1$  or the cached sequence  $(E_k)_k \subseteq \mathbb{G}_T$ . Choosing the public indices as  $\alpha_i = \gamma\omega^{i-1}$ , where  $\omega$  is a primitive  $\ell$ -th root of unity and  $\gamma$  is chosen so that every  $\alpha_i \notin \{0, -1\}$ , we can write  $\mathcal{C}_i(X)$  as

$$\mathcal{C}_i(X) = \gamma^{-1}\omega^{1-i} \sum_{d \neq 0} \frac{X_{i+d}}{\omega^d - 1},$$

with indices interpreted modulo  $\ell$ . Thus, the values  $(\mathcal{C}_i(X))_{i \in [\ell]}$  form a cyclic convolution and can be computed using an FFT in  $O(\ell \log \ell)$  group operations. To see the computation explicitly, define the scalar kernel  $h_0 := 0$  and  $h_d := (\omega^d - 1)^{-1}$  for  $d \neq 0$ . Up to the public factor  $\gamma^{-1}\omega^{1-i}$ , the vector  $(\mathcal{C}_i(X))_{i \in [\ell]}$  is the cyclic convolution of  $(X_i)_i$  with  $(h_d)_d$  after reversing the kernel. It is therefore obtained by taking a group FFT of  $(X_i)_i$ , multiplying its coordinates by the ordinary field FFT of the fixed kernel, and applying an inverse group FFT; the procedure is run once with  $X_i = \text{ct}_i[1] \in \mathbb{G}_1$  and once with  $X_i = E_i \in \mathbb{G}_T$ . This reduces the non-pairing work of decryption from  $O(\ell^2)$  to  $O(\ell \log \ell)$  without changing the pairing complexity.

Finally, note that the submitted pre-decryption keys can also be batch verified by sampling  $\alpha_j \leftarrow \mathbb{F}_p$  for  $j \in T$ , and checking

$$e\left(\sum_{j \in T} \alpha_j \text{sbk}_j, [1]_2\right) = \sum_{i=1}^{\ell} e\left(\text{ct}_i[1], \sum_{j \in T} \alpha_j \text{vk}_{j,i}\right)$$

**Theorem 5 (Correctness and robustness).** *Construction 5 is correct and satisfies robustness.*

*Proof.* Fix an authorized set  $T'$  of accepted pre-decryption keys and an index  $i \in [\ell]$ . Write  $\text{ct}_k[1] = [r_k]_1$  and set

$$z_j := \sum_{w \in \Omega_j} \lambda_{T',w} z_w \quad (j \in T')$$

Since  $\Omega_{T'}$  contains more than  $t$  evaluation points and  $Z$  has degree at most  $t$ , Lagrange interpolation gives

$$\sum_{j \in T'} z_j = \sum_{w \in \Omega_{T'}} \lambda_{T',w} z_w = z. \quad (12)$$

Now,  $L_{j,i}$  is given by

$$\begin{aligned} & e\left(\text{sbk}_j - \sum_{\substack{k \in [\ell] \\ k \neq i}} \frac{\text{ct}_k[1]}{\alpha_k - \alpha_i}, \sum_{w \in \Omega_j} \lambda_{T',w} \text{dk}_{w,i}[1]\right) + \sum_{\substack{k \in [\ell] \\ k \neq i}} e\left(\frac{\text{ct}_k[1]}{\alpha_k - \alpha_i}, \sum_{w \in \Omega_j} \lambda_{T',w} \text{dk}_{w,k}[1]\right) \\ &= \left[ z_j g_{j,i} \left( \sum_{k=1}^{\ell} r_k g_{j,k} - \sum_{\substack{k \in [\ell] \\ k \neq i}} \frac{r_k}{\alpha_k - \alpha_i} \right) + z_j \sum_{\substack{k \in [\ell] \\ k \neq i}} \frac{r_k g_{j,k}}{\alpha_k - \alpha_i} \right]_T \end{aligned}$$

Observe that the coefficient of  $z_j r_k$  for  $k \neq i$  is

$$g_{j,i} g_{j,k} - \frac{g_{j,i}}{\alpha_k - \alpha_i} + \frac{g_{j,k}}{\alpha_k - \alpha_i} = 0$$

by the partial fraction identity at Equation (11). Consequently,

$$L_{j,i} = [r_i z_j g_{j,i}^2]_T = \sum_{w \in \Omega_j} [r_i \lambda_{T',w} z_w g_{j,i}^2]_T. \quad (13)$$

Finally, using the definition of  $\mathbf{dk}_{w,i}[2]$ , partitioning the points in  $\Omega_{T'}$  by owner, and using Equation (12), we obtain

$$e\left(\mathbf{ct}_i[1], \sum_{w \in \Omega_{T'}} \lambda_{T',w} \mathbf{dk}_{w,i}[2]\right) = \left[r_i z \left(\frac{\beta_{-1}}{\alpha_i + 1} + \frac{\beta_0}{\alpha_i}\right)\right]_T + \sum_{j \in T'} [r_i z_j g_{j,i}^2]_T.$$

Therefore, subtracting  $\sum_{j \in T'} L_{j,i}$  from the term above, we derive the one-time pad

$$\alpha_i \left( (\alpha_i + 1) \left[ r_i z \left( \frac{\beta_{-1}}{\alpha_i + 1} + \frac{\beta_0}{\alpha_i} \right) \right]_T - [r_i z (\beta_{-1} + \beta_0)]_T \right) = [r_i z \beta_0]_T,$$

which proves correctness.

It remains to prove robustness. To show accurate blaming, observe that a correctly generated pre-decryption key always passes its verification check:

$$e(\mathbf{sbk}_j, [1]_2) = e\left(\sum_{i=1}^{\ell} g_{j,i} \mathbf{ct}_i[1], [1]_2\right) = \sum_{i=1}^{\ell} e(\mathbf{ct}_i[1], g_{j,i}) = \sum_{i=1}^{\ell} e(\mathbf{ct}_i[1], \mathbf{vk}_{j,i}) \quad (14)$$

Finally, by Equation (14), every accepted set of pre-decryption keys  $(\mathbf{sbk}_j)_{j \in T}$ , for which the decryption algorithms outputs a non- $\perp$  value satisfies  $\mathbf{sbk}_j = \sum_{i=1}^{\ell} g_{j,i} \cdot \mathbf{ct}_i[1]$  for all  $j$ . Thus, any two sets of accepted pre-decryption keys for the same batch of ciphertexts are equal, and give the same set of decrypted messages because decryption is a deterministic operation. Accurate blaming and consistency together imply the robustness of Construction 5.  $\square$

#### 4.1 Analysis of Construction 5

We use the following weighted analog of  $q$ -QuadRankDef [12, Assumption 2] used by the unweighted version of Construction 5.

**Assumption 6** (Weighted Quadratic Rank Deficiency, WQuadRankDef). *Given parameters  $(N, W, (w_j)_{j \in [N]}, (\Omega_j)_{j \in [N]}, t, \ell, (\alpha_i)_{i \in [\ell]})$ , define the two-stage experiment  $\text{WQRD}_{\mathcal{B}}^b(1^\lambda)$  for  $b \in \{0, 1\}$  and distinguisher  $\mathcal{B} = (\mathcal{B}_1, \mathcal{B}_2)$ , as follows. The challenger samples the bilinear groups,  $z, a_1, \dots, a_t, \beta_{-1}, \beta_0 \xleftarrow{\$} \mathbb{F}_p$ , subject to  $\beta_0 \neq 0$  and  $\beta_{-1} + \beta_0 \neq 0$ , and sets*

$$Z(X) = z + \sum_{s=1}^t a_s X^s, \quad z_w = Z(w).$$

For each  $j \in [N]$ , it samples  $\rho_j \leftarrow \mathbb{F}_p$  subject to the conditions in Construction 5, and defines

$$g_{j,i} = \frac{1}{\rho_j + \alpha_i}, \quad u_{j,i} = g_{j,i}^2 + \frac{\beta_{-1}}{\alpha_i + 1} + \frac{\beta_0}{\alpha_i}.$$

It computes

$$\begin{aligned} \mathbf{ek} &= [z\beta_0]_T, \\ \mathbf{vk}_{j,i} &= [g_{j,i}]_2, \\ \mathbf{dk}[0] &= [z(\beta_{-1} + \beta_0)]_2, \\ \mathbf{dk}_{w,i}[1] &= [z_w g_{\text{own}(w),i}]_2, \\ \mathbf{dk}_{w,i}[2] &= [z_w u_{\text{own}(w),i}]_2, \\ H_{j,i} &= [g_{j,i}]_1. \end{aligned}$$

Let  $\mathbf{pub}$  contain  $(\Omega_j)_{j \in [N]}$ ,  $\mathbf{ek}$ , all verification keys, and all decryption key elements. In the first stage,

$$(\mathcal{C}, \mathbf{st}) \leftarrow \mathcal{B}_1(\mathbf{pub}, (H_{j,i})_{j \in [N], i \in [\ell]}).$$

The experiment returns 0 if  $\text{wt}(\mathcal{C}) > t$ . Otherwise, it samples  $r, u \leftarrow \mathbb{F}_p$ , sets

$$X = [r]_1, \quad Y_0 = [rz\beta_0]_T, \quad Y_1 = [u]_T,$$

sets  $\mathbf{sk}_j = \rho_j$  for every  $j \in [N]$ , and returns the output of

$$\mathcal{B}_2(\mathbf{st}, (\mathbf{sk}_j)_{j \in \mathcal{C}}, X, Y_b).$$

The advantage is

$$\text{Adv}_{\mathcal{B}}^{\text{WQuadRankDef}}(\lambda) = |\Pr[\text{WQRD}_{\mathcal{B}}^0(1^\lambda) = 1] - \Pr[\text{WQRD}_{\mathcal{B}}^1(1^\lambda) = 1]|.$$

The WQuadRankDef assumption states that this advantage is negligible for every PPT distinguisher and every polynomially bounded choice of valid parameters.

As before, the elements  $(H_{j,i})_{j \in [N], i \in [\ell]}$  are used by the simulation and are given to the distinguishing adversary, but are not public parameters of Construction 1. The assumption is falsifiable, as both distributions in the experiment are efficiently samplable.

**Theorem 7 (Static WB-IND-CCA Security).** *Suppose Assumption 6 holds and  $\Pi_{\text{DL}}$  is complete, zero knowledge, and straightline, simulation extractable for  $R_{\text{DL}}$ . Then, Construction 5 is WB-IND-CCA secure.*

*Proof.* Let  $\mathcal{A}$  be the WB-IND-CCA adversary (Definition 3). We consider the following games, and write  $S_k$  for the event that  $\mathcal{A}$  outputs the challenge bit in Game  $k$ .

**Game 0.** This is the real WB-IND-CCA game. Hence

$$\left| \Pr[S_0] - \frac{1}{2} \right| = \text{Adv}_{\text{WPFE}, \mathcal{A}}^{\text{WB-IND-CCA}}(\lambda).$$

**Game 1.** At setup, we replace the NIZK CRS by

$$(\text{crs}, \text{td}_{\text{sim}}, \text{td}_{\text{ext}}) \leftarrow \Pi_{\text{DL}}.\text{SimSetup}(1^\lambda),$$

and generate the proof within the challenge ciphertext using  $\Pi_{\text{DL}}.\text{SimProve}$ . The zero-knowledge property implies that there is a negligible function  $\epsilon_{\text{zk}}$  such that

$$|\Pr[S_0] - \Pr[S_1]| \leq \epsilon_{\text{zk}}.$$

**Game 2.** We change the implementation of the pre-decryption oracle. Given a query  $(\text{ct}_i)_{i \in I}$ , where  $\text{ct}_i = (\text{ct}_i[1], \text{ct}_i[2], \pi_i)$  and  $I \subseteq [\ell]$ , the oracle first performs the prescribed verification and returns  $\perp$  if a proof is invalid. It then computes  $r_i \leftarrow \Pi_{\text{DL}}.\text{Ext}(\text{crs}, \text{td}_{\text{ext}}, (\text{ct}_i[1], \text{ct}_i[2]), \pi_i)$  for every accepting ciphertext, and returns

$$\widehat{\text{sbk}}_j = \sum_{i \in I} r_i H_{j,i} = \sum_{i \in I} g_{j,i} \text{ct}_i[1].$$

for every  $j \notin \mathcal{C}$ . Thus, the simulated response is the prescribed output of  $\text{PreDec}$ .

Before the challenge, the simulator has returned no statement-proof pair. After the challenge, the only such pair is the challenge pair  $((\text{ct}^*[1], \text{ct}^*[2]), \pi^*)$ . The WB-IND-CCA game rejects a query containing the exact challenge ciphertext. Hence, every pair on which the extractor is invoked is outside the simulator's output set, including the pairs which use  $(\text{ct}^*[1], \text{ct}^*[2])$  as the statement along with a fresh proof. Then, by the simulation-extractability of the NIZK and a union bound over all queried ciphertexts, the probability that  $\text{ct}_i[1] \neq [r_i]_1$  for an extracted  $r_i \leftarrow \Pi_{\text{DL}}.\text{Ext}(\text{crs}, \text{td}_{\text{ext}}, (\text{ct}_i[1], \text{ct}_i[2]), \pi_i)$  is bounded by a negligible function  $\epsilon_{\text{ext}}$ . Therefore, it follows that

$$|\Pr[S_1] - \Pr[S_2]| \leq \epsilon_{\text{ext}}.$$

**Game 3.** In Game 3, we replace the challenge pad  $[r^* z \beta_0]_T$  by an independent uniformly random element. A distinguisher  $\mathcal{B}$  for Assumption 6 simulates Game 2 using the handles  $(H_{j,i})_{j \in [N], i \in [\ell]}$ . It forwards the corrupted set  $\mathcal{C}$  chosen by  $\mathcal{A}$  and gives the returned secret keys  $(g_{j,i})_{i \in [\ell]}, j \in \mathcal{C}$ , to  $\mathcal{A}$ . On challenge messages  $m_0, m_1$ , it samples  $\mu \leftarrow_{\$} \{0, 1\}$ , and using its assumption challenge  $(X, Y_b)$ , returns

$$\text{ct}^* = (X, m_\mu + Y_b, \Pi_{\text{DL}}.\text{SimProve}(\text{crs}, \text{td}_{\text{sim}}, (X, m_\mu + Y_b))).$$

It outputs 1 if and only if  $\mathcal{A}$  outputs  $\mu$ . Now, if  $b = 0$ , Game 3 is identical to Game 2. Consequently,

$$|\Pr[S_2] - \Pr[S_3]| \leq \text{Adv}_{\mathcal{B}}^{\text{WQuadRankDef}}(\lambda).$$

In Game 3, the second component of the challenge ciphertext is uniform and independent of  $\mu$ , so  $\Pr[S_3] = 1/2$ . Combining the games gives

$$\text{Adv}_{\text{WPFE}, \mathcal{A}}^{\text{WB-IND-CCA}}(\lambda) \leq \text{Adv}_{\mathcal{B}}^{\text{WQuadRankDef}}(\lambda) + \epsilon_{\text{zk}} + \epsilon_{\text{ext}},$$

which is negligible.  $\square$

**Lemma 2 (Bilinear generic group hardness).** *Assumption 6 holds in the generic bilinear group model, except with probability  $O(Q^2(3N\ell+3)/p)$ , where  $Q$  denotes the total number of generic group handles occurring in the experiment.*

*Proof.* Fix the set  $\mathcal{C} \subseteq [N]$  returned in the first stage and reveal  $(g_{i,j})_{i \in [\ell], j \in \mathcal{C}}$  to the adversary. In the symbolic game, we treat

$$r, z, a_1, \dots, a_t, \beta_{-1}, \beta_0, (\rho_j)_{j \notin \mathcal{C}}$$

as algebraically independent indeterminates. Any formal  $\mathbb{G}_T$  expression that contains  $r$  has the form  $rL + L'$ , where  $L$  lies in the linear span of the public  $\mathbb{G}_2$  exponents, and  $L'$  does not contain  $r$ . It is therefore enough to prove that  $z\beta_0$  is not in the linear span of the Laurent polynomials

$$\begin{aligned} 1, \quad g_{j,i} &:= \frac{1}{\rho_j + \alpha_i}, \quad z(\beta_{-1} + \beta_0), \\ Z(w)g_{\text{own}(w),i}, \\ Z(w) \left( g_{\text{own}(w),i}^2 + \frac{\beta_{-1}}{\alpha_i + 1} + \frac{\beta_0}{\alpha_i} \right), \end{aligned}$$

over  $j \in [N]$ ,  $i \in [\ell]$ , and  $w \in \Omega_{[N]}$ .

Consider a party  $j \notin \mathcal{C}$  and fix  $i \in [\ell]$ . The contribution of all terms  $\text{dk}_{w,i}[2]$  with  $w \in \Omega_j$  has the form

$$\left( \sum_{w \in \Omega_j} c_{w,i} Z(w) \right) \left( g_{j,i}^2 + \frac{\beta_{-1}}{\alpha_i + 1} + \frac{\beta_0}{\alpha_i} \right)$$

for coefficients  $c_{w,i} \in \mathbb{F}_p$ . At the pole  $g_{j,i} = 1/(\rho_j + \alpha_i)$  (denoted by  $\rho_j = -\alpha_i$ ), the coefficient of the quadratic term  $g_{j,i}^2$  is

$$\sum_{w \in \Omega_j} c_{w,i} Z(w).$$

However, the target  $z\beta_0$  has no pole in  $\rho_j$ , and the terms associated with indices  $i' \neq i$  have poles  $\rho_j = -\alpha_{i'}$  at different locations. The displayed coefficient must therefore vanish, making the contribution from linear combinations of the  $\text{dk}_{w,i}[2]$  terms for  $\text{own}(w) \notin \mathcal{C}$  zero.

After eliminating these terms, the remaining expressions having a pole at  $\rho_j = -\alpha_i$  are of the form

$$g_{j,i} \left( c_i + \sum_{w \in \Omega_j} b_{w,i} Z(w) \right),$$

where  $c_i, b_{w,i} \in \mathbb{F}_p$ . Its coefficient must also vanish identically, making the contribution from linear combinations of the  $\text{dk}_{w,i}[1]$  and  $[g_{j,i}]_2$  terms for  $\text{own}(w) \notin \mathcal{C}$

zero. Consequently, terms belonging to uncorrupted parties cannot contribute to a pole-free expression for  $z\beta_0$ .

For every corrupted party  $j \in \mathcal{C}$ , the revealed secret key  $\rho_j$  determines all

$$g_{j,i} = (\rho_j + \alpha_i)^{-1}.$$

We may therefore grant the adversary the larger span

$$1, \quad Z(w), \quad Z(w)\beta_{-1}, \quad Z(w)\beta_0 \quad (w \in \Omega_{\mathcal{C}}),$$

in addition to  $z(\beta_{-1} + \beta_0)$ . For contradiction, suppose  $z\beta_0$  belongs to this enlarged span. Then, there exist coefficients  $c_0, c_1, (e_w)_w, (b_w)_w, (d_w)_w \in \mathbb{F}_p$  such that

$$\begin{aligned} z\beta_0 &= c_0 + c_1 z(\beta_{-1} + \beta_0) \\ &\quad + \sum_{w \in \Omega_{\mathcal{C}}} (e_w Z(w) + b_w Z(w)\beta_{-1} + d_w Z(w)\beta_0). \end{aligned}$$

Comparing the coefficients of  $\beta_{-1}$  and  $\beta_0$  gives

$$0 = cz + \sum_{w \in \Omega_{\mathcal{C}}} b_w Z(w), \quad z = cz + \sum_{w \in \Omega_{\mathcal{C}}} d_w Z(w). \quad (15)$$

If  $c \neq 0$ , the first equality expresses  $z$  as a linear combination of the evaluations  $(Z(w))_{w \in \Omega_{\mathcal{C}}}$ . If  $c = 0$ , the second equality does so. Now, suppose that

$$z = \sum_{w \in \Omega_{\mathcal{C}}} \mu_w Z(w)$$

for coefficients  $\mu_w \in K$ . Since

$$Z(w) = z + \sum_{s=1}^t a_s w^s,$$

comparison of the coefficients of  $a_s$ , for  $s \in [t]$ , gives

$$\sum_{w \in \Omega_{\mathcal{C}}} \mu_w w^s = 0.$$

The points in  $\Omega_{\mathcal{C}}$  are distinct and nonzero, and  $|\Omega_{\mathcal{C}}| = \text{wt}(\mathcal{C}) \leq t$ . Therefore, the corresponding Vandermonde matrix has full column rank, and  $\mu_w = 0$  for every  $w \in \Omega_{\mathcal{C}}$ . Comparing the coefficient of  $z$  then gives the desired contradiction. Hence, it follows that  $z\beta_0$  is outside the formal  $\mathbb{G}_2$  span.

Finally, the adversary can distinguish the symbolic and real games only if two distinct formal expressions in the symbolic game receive the same value in real game. Let

$$D := \prod_{j \notin \mathcal{C} \ i \in [\ell]} (\rho_j + \alpha_i) \quad \text{and} \quad M := (N - |\mathcal{C}|)\ell$$

The denominators of the exponents represented in  $\mathbb{G}_1$  and  $\mathbb{G}_2$  divide  $D$  and  $D^2$ , respectively, since the public encodings contain both  $g_{j,i}$  and  $g_{j,i}^2$ . A pairing therefore produces a rational function whose denominator divides  $D^3$  and whose numerator has degree at most  $3M+3 \leq 3(N\ell+1)$ . Hence, by the Schwartz–Zippel lemma and a union bound over at most  $O(Q^2)$  pairs of expressions, after clearing the denominators of the Laurent polynomials, the adversary can distinguish the symbolic and real games with probability at most  $O(Q^2(3N\ell+3)/p)$ , which gives the stated bound.  $\square$

#### 4.2 Cost analysis for Construction 5

In Table 3, we give the storage and computational costs for the partial fraction-based weighted BTE scheme.

#### 4.3 Distributed Key Generation for Construction 5

The public indices  $\alpha_1, \dots, \alpha_\ell \in \mathbb{F}_p \setminus \{0, -1\}$  are fixed before the protocol begins and are included in the public DKG context. In particular, they are not sampled by any DKG participant. The change from  $p_{-1}(x), p_0(x)$  to the independently generated scalars  $\beta_{-1}, \beta_0$  makes the setup amenable to a two-step DKG. **[Sasha: Where is the change from? From the original PFE paper?]** The first step generates each party’s secret shift  $\rho_j$ , public encodings of the derived values

$$g_{j,i} = (\rho_j + \alpha_i)^{-1},$$

and the two global scalars. The second step generates the  $\leq t$  polynomial  $Z$ .

We use the same authenticated reliable-broadcast and total-order-agreement primitives as in the DKG for Construction 1. Every broadcast is signed and bound to the session identifier, the public parameters, the sender, and the complete transcript. The proofs of knowledge below are straight-line extractable by assumption.

*Step 1: Generation of the local shifts and global scalars.* Party  $P_j$  samples

$$\rho_j, \beta_{-1}^{(j)}, \beta_0^{(j)} \xleftarrow{\$} \mathbb{F}_p$$

and sets

$$\bar{R}_j := [\rho_j]_1, \quad B_{j,a} := [\beta_a^{(j)}]_2 \quad (a \in \{-1, 0\}).$$

For every  $i \in [\ell]$ , it derives

$$g_{j,i} := (\rho_j + \alpha_i)^{-1}$$

and computes

$$G_{j,i}^{(1)} := [g_{j,i}]_1, \quad G_{j,i}^{(2)} := [g_{j,i}]_2, \quad Q_{j,i} := [g_{j,i}^2]_2.$$

Party  $P_j$  reliably broadcasts these values, extractable proofs of knowledge of  $\rho_j, \beta_{-1}^{(j)}, \beta_0^{(j)}$ , and a signature on its complete Step-1 transcript. It retains

$$\text{sk}_j := \rho_j$$

as its secret key. It may additionally cache  $(g_{j,i})_{i \in [\ell]}$  as derived local state.

A recipient verifies the proofs and signature and, for every  $i \in [\ell]$ , checks

$$e(\bar{R}_j + \alpha_i[1]_1, G_{j,i}^{(2)}) = [1]_T, \quad (16)$$

$$e(G_{j,i}^{(1)}, [1]_2) = e([1]_1, G_{j,i}^{(2)}), \quad (17)$$

$$e(G_{j,i}^{(1)}, G_{j,i}^{(2)}) = e([1]_1, Q_{j,i}). \quad (18)$$

The recipient also checks that every  $\bar{R}_j + \alpha_i[1]_1$  is nonzero and that these elements are pairwise distinct across all accepted pairs  $(j, i)$ .

The parties use total-order agreement to select the first valid set  $J \subseteq [N]$  satisfying  $\text{wt}(J) > t$ . This fixes the active owner set and the evaluation domain

$$\Omega_J := \bigcup_{j \in J} \Omega_j.$$

No Step-2 message is processed before  $J$  is fixed.

For  $a \in \{-1, 0\}$ , define

$$B_a := \sum_{j \in J} B_{j,a} = [\beta_a]_2, \quad \beta_a := \sum_{j \in J} \beta_a^{(j)}.$$

For every  $j \in J$  and  $i \in [\ell]$ , define

$$U_{j,i} := Q_{j,i} + \frac{B_{-1}}{\alpha_i + 1} + \frac{B_0}{\alpha_i}. \quad (19)$$

The parties restart Step 1 if

$$B_0 = [0]_2 \quad \text{or} \quad B_{-1} + B_0 = [0]_2.$$

These checks impose the conditions  $\beta_0 \neq 0$  and  $\beta_{-1} + \beta_0 \neq 0$  without revealing either aggregate scalar.

*Step 2: Generation of the sharing polynomial.* After  $J$  is fixed, each dealer  $P_h$  samples a uniformly random polynomial

$$Z^{(h)}(X) = \sum_{s=0}^t z_s^{(h)} X^s$$

and broadcasts coefficient commitments

$$A_{h,s} := [z_s^{(h)}]_1 \quad (s \in \{0, \dots, t\}),$$

together with extractable proofs of knowledge of all coefficients and a signature on its complete Step-2 transcript. For  $w \in \{0\} \cup \Omega_J$ , define

$$A_{h,w} := \sum_{s=0}^t w^s A_{h,s} = [Z^{(h)}(w)]_1.$$

For every  $w \in \Omega_J$  and  $i \in [\ell]$ , dealer  $P_h$  broadcasts

$$\begin{aligned} D_{h,w,i}[1] &:= Z^{(h)}(w) G_{\text{own}(w),i}^{(2)}, \\ D_{h,w,i}[2] &:= Z^{(h)}(w) U_{\text{own}(w),i}, \\ V_h &:= z_0^{(h)} (B_{-1} + B_0). \end{aligned}$$

A recipient accepts the transcript only if, for every  $w$  and  $i$ ,

$$e(A_{h,w}, G_{\text{own}(w),i}^{(2)}) = e([1]_1, D_{h,w,i}[1]), \quad (20)$$

$$e(A_{h,w}, U_{\text{own}(w),i}) = e([1]_1, D_{h,w,i}[2]), \quad (21)$$

$$e(A_{h,0}, B_{-1} + B_0) = e([1]_1, V_h). \quad (22)$$

The parties use total-order agreement to select the first set  $I \subseteq [N]$  of valid dealer transcripts satisfying  $\text{wt}(I) > t$ . They aggregate the accepted transcripts componentwise. For  $w \in \Omega_J$ ,  $i \in [\ell]$ , and  $b \in \{1, 2\}$ , set

$$A_s := \sum_{h \in I} A_{h,s}, \quad A_w := \sum_{h \in I} A_{h,w}, \quad (23)$$

$$\text{dk}_{w,i}[b] := \sum_{h \in I} D_{h,w,i}[b], \quad \text{dk}[0] := \sum_{h \in I} V_h, \quad (24)$$

$$\text{ek} := e(A_0, B_0), \quad \text{vk}_{j,i} := G_{j,i}^{(2)}. \quad (25)$$

For every  $j \in J$ , party  $P_j$  retains

$$\text{sk}_j = \rho_j.$$

After the DKG, the parties may relabel the active owner set  $J$  as the committee  $[N]$  used in Construction 5.

**Theorem 8 (Correctness of the partial-fraction DKG).** *Assume sound extractable proofs of knowledge, unforgeable signatures, reliable broadcast, and total-order agreement as above. Except with negligible probability, every terminating execution of the protocol produces common keys having the algebraic form specified by Setup in Construction 5, over the active domain  $\Omega_J$ .*

*Proof.* Condition on the event that no signature forgery or violation of proof soundness occurs. Total-order agreement implies that all honest parties select the same sets  $J$  and  $I$  and the same signed transcripts.

For every  $j \in J$ , extraction fixes scalars

$$\rho_j, \beta_{-1}^{(j)}, \beta_0^{(j)}$$

underlying  $\overline{R}_j, B_{j,-1}, B_{j,0}$ . Write

$$G_{j,i}^{(2)} = [a_{j,i}]_2.$$

Equation (16) and nondegeneracy give

$$(\rho_j + \alpha_i)a_{j,i} = 1,$$

and hence

$$a_{j,i} = (\rho_j + \alpha_i)^{-1} = g_{j,i}.$$

Equation (17) therefore forces

$$G_{j,i}^{(1)} = [g_{j,i}]_1,$$

and Equation (18) forces

$$Q_{j,i} = [g_{j,i}^2]_2.$$

The nonzero and pairwise-distinctness checks give exactly the conditions on the denominators required by Setup.

By componentwise aggregation,

$$B_a = [\beta_a]_2, \quad \beta_a = \sum_{j \in J} \beta_a^{(j)} \quad (a \in \{-1, 0\}).$$

Consequently, Equation (19) gives

$$U_{j,i} = [u_{j,i}]_2, \quad u_{j,i} = g_{j,i}^2 + \frac{\beta_{-1}}{\alpha_i + 1} + \frac{\beta_0}{\alpha_i}. \quad (26)$$

The restart checks ensure

$$\beta_0 \neq 0 \quad \text{and} \quad \beta_{-1} + \beta_0 \neq 0.$$

Extraction from an accepted Step-2 proof fixes a polynomial  $Z^{(h)} \in \mathbb{F}_p[X]$  of degree at most  $t$  whose coefficients open the commitments  $(A_{h,s})_s$ . Thus

$$A_{h,w} = [Z^{(h)}(w)]_1$$

for every  $w \in \{0\} \cup \Omega_J$ . Equations (20)–(22) and nondegeneracy imply

$$\begin{aligned} D_{h,w,i}[1] &= [Z^{(h)}(w)g_{\text{own}(w),i}]_2, \\ D_{h,w,i}[2] &= [Z^{(h)}(w)u_{\text{own}(w),i}]_2, \\ V_h &= [Z^{(h)}(0)(\beta_{-1} + \beta_0)]_2. \end{aligned}$$

Define

$$Z(X) := \sum_{h \in I} Z^{(h)}(X), \quad z := Z(0), \quad z_w := Z(w).$$

Then  $\deg(Z) \leq t$ , and componentwise aggregation gives

$$\begin{aligned} A_0 &= [z]_1, & A_w &= [z_w]_1, \\ \mathbf{dk}_{w,i}[1] &= [z_w g_{\text{own}(w),i}]_2, & \mathbf{dk}_{w,i}[2] &= [z_w u_{\text{own}(w),i}]_2, \\ \mathbf{dk}[0] &= [z(\beta_{-1} + \beta_0)]_2, & \mathbf{ek} &= e([z]_1, [\beta_0]_2) = [z\beta_0]_T. \end{aligned}$$

Together with

$$\mathbf{sk}_j = \rho_j, \quad \mathbf{vk}_{j,i} = [g_{j,i}]_2, \quad g_{j,i} = (\rho_j + \alpha_i)^{-1},$$

these are precisely the keys specified by Setup. Since every honest party derives them from the same accepted transcripts, all honest outputs agree.  $\square$

The first step communicates  $O(N\ell)$  group elements. If all  $N$  parties act as dealers, the second step communicates

$$O(N(W\ell + t))$$

group elements, where  $W = |\Omega_J|$ . The conditions  $\text{wt}(J) > t$  and  $\text{wt}(I) > t$  ensure that each accepted set contains an honest party whenever the corrupted weight is at most  $t$ .

The theorem above establishes algebraic correctness of the generated keys. When security is claimed for DKG-generated parameters, the public DKG transcript must be included explicitly in the public distribution of the corresponding transcript-augmented version of Assumption 6.

## 5 Experiments

We next report performance results from our experiments. All experiments were run on an Apple MacBook Pro with an M4 processor and 24 GB of RAM. We implemented our experiments in Rust and used the `blst` library's [27] implementation of fields, curves and pairings. We also reimplemented relevant prior schemes from the literature for comparison purposes, using similarly optimized implementations of curves, fields and pairings. Code for our experiments is available in [6].

We benchmarked the schemes with an authorization threshold of  $1/2$ . We chose the distribution of weights to approximate Solana's share distribution [7]. We used Aptos' approach [9] to compute the share assignments and total required numbers of shares. For our experiments, we chose total weights of 764; 1,580; 3,063; and 6,489 corresponding to different qualities of approximation. These are the lowest total weight counts for an approximation of Solana's stake distribution such that no committee with total stake less than  $1/2 - \epsilon$  is unauthorized, and any

committee with total stake more than  $1/2 + \epsilon$  is authorized for  $\epsilon \in 1/8, 1/16, 1/32$ , and  $1/64$  respectively. We describe the parameter choices and Aptos’ algorithm in greater depth in Appendix A.

To contextualize the performance thresholds for our work, we focus on Solana again. Solana has block times of 400ms [8]. Solana nodes are run on servers with many cores. Based on personal communications with Solana developers, 12 cores of a powerful server is a reasonable allocation to expect for this task, and decryption (the operation on the “hot path” of consensus) should take a fraction of 400ms. Based on conversations with the developers, batch sizes of 32 is a reasonable target for number of concurrent proposals, and thus the batch size to target [25].

Furthermore, to give a baseline for DKG performance, Solana has “epochs” which last 2 days [8]. The set of Solana validators is fixed for each epoch, so a DKG must be run for each epoch. As previously, DKG should run much faster than 2 days.

### 5.1 Performance results for Constructions 1 and 5

[Sasha: TODO for me: Refactor these tables if we can to mention the precomputable phases.]

In Table 5, we give performance numbers for our two weighted batch encryption schemes. We show the performance for the main parts of the weighted decryption flow. We label Construction 1 as WBTX and Construction 5 as WPFE. In Table 6, we show how the performance of the two schemes varies, across different batch sizes. As previously mentioned, for the use case of encrypting Solana proposals, 32 is the most relevant batch size.

We give the other costs of the schemes in Table 7. We give the encryption time, and the sizes of various public keys.

### 5.2 Comparison to prior work

We show the performance of 3 prior schemes from the literature. Concretely, we reimplement the index dependent weighted batch threshold encryption scheme of [1], labeled BEAT++. We also implement two prior (unweighted) batch threshold encryption schemes. We label the scheme of BTX and Policharla [4,24] as BTX and the scheme of [12] as PFE.<sup>3</sup>

Implementations for BEAT++ and BTX were not available publicly at the time of writing. We reimplemented them using the same collection of primitives and level of optimization as our schemes, which made the comparisons more fair. BTX uses AVX-512-optimized primitives. While there was an implementation of the [24] concurrent paper proposing an equivalent scheme to BTX, and an implementation of the partial fraction-based BTE scheme of [12], our reimplementations of these schemes was faster than the respective schemes, so

<sup>3</sup> Meaning Partial Fraction Encryption.

this actually makes comparisons more fair. An independent contribution of our work is the fastest open source reimplementation of the scheme of [4,24].

For BEAT++, we ran our implementation with the same per-party weights and batch sizes as our scheme. As can be seen BEAT++ was about 30-40% faster than our scheme at these batch sizes. Their scheme has a somewhat simpler algebraic structure than our scheme because it is index-dependent.

For BTX and PFE, we ran the schemes with parameters that one would use with “naive virtualization”, meaning that for a party assigned weight  $w_j$ , it would simply be given  $w_j$  independent keys in a non-weighted batch threshold encryption scheme. These schemes were 3-5 $\times$  faster than our scheme. This corresponds to their simpler algebraic structure. However, as mentioned previously, these schemes only support DKGs via generic MPCs, have similar key sizes and other performance metrics, and do not have pre-decryption shares which are succinct in the amount of weight assigned to an individual party. For instance, the parties with the most stake at the largest settings of  $W$  of 6,488 had total weights of more than 100-200, so this has an impact on the “hot-path” communication for threshold decryption.

To conclude, the prior schemes are faster across the board, though this is to be expected because they have fewer features and have simpler structures which require less algebraic work.

## 6 Related work

We discuss the main relevant lines of prior research to this work.

**Encrypted mempools/MEV.** Public blockchain mempools reveal pending transactions before their ordering is finalized, enabling front-running and other forms of maximal extractable value (MEV) [19]. Encrypted mempools seek to mitigate these attacks by hiding transaction contents until their order has been fixed. This setting motivates threshold encryption schemes which remain efficient for large batches of transactions and whose access structures reflect each validator’s stake [22].

**Batch- and epoch-labelled BTE.** Choudhuri et al. [17] introduced the idea of batch threshold encryption (BTE), which lets each party in a threshold encryption scheme publish a short pre-decryption share to decrypt an entire selected batch of ciphertexts while ciphertexts outside that batch remain private. Subsequent works improved on this work by supporting reusable setups or building related batched identity- and tag-based schemes [18,5,26]. These constructions still associate every ciphertext with some sort of batch-specific information, so a ciphertext omitted from its intended batch may become “stranded”.

**Epochless, index-dependent BTE.** BEAT-MEV [15] removes batch labels and allows ciphertexts to be reused across batches. However, its encryption requires an index from a fixed public domain, and a decryptable batch must contain at most one ciphertext for each index. Dealing with index collisions requires additional communication or leads to issues with censorship resistance. Subsequent works improve on this approach [14,1].

**Weighted threshold cryptography.** Proof-of-stake systems motivate cryptographic schemes with weighted access structures, since the influence of a validator is determined by its stake. Ferveo [11] supports (non batched) weighted threshold decryption while compressing the virtual decryption shares for one ciphertext into a single group element per party. Agarwal et al. [2] give the first weighted BTE scheme where a party’s pre-decryption communication is independent of its assigned weight, and reduce decryption from quadratic to quasilinear complexity, though its encryption is still index-dependent.

**Epochless, index-free BTE.** Recent works construct pairing-based Batch Threshold Encryption schemes without indices or epoch-specific labels. BTX [4] and Policharla [24] give the same BTE construction with short ciphertexts and quasilinear decryption. Concurrently, Boneh et al. [12] adapt partial fraction techniques [23] to construct an epochless and index-free BTE scheme. Both schemes achieve similar properties, though the BTX construction is concretely more efficient. We use these schemes as the starting points for Construction 1 and Construction 5 respectively. We extend them to support weighted access structures and a simple DKG procedure.

## 7 Conclusion

We introduce two constructions for weighted batch threshold encryption. Starting from the BTX construction of Agarwal et al. and the partial fraction construction of Boneh et al., we build two schemes where each party releases only one group element for an entire ciphertext batch as part of decryption, independent of its assigned weight. Together, they are the first weighted batch threshold encryption schemes with batch-independent, index-free ciphertexts. For the more efficient BTX-based construction, we additionally provide a specialized distributed key-generation protocol that avoids a general-purpose MPC setup. Our Rust implementation further demonstrates that these techniques are practical at parameter sizes relevant to encrypted mempools. For remaining future work:

[Sasha: Finish the discussion of limitations and future work when the paper is closer to completion.]

## References

1. Agarwal, A., Babel, K., Das, S., Gilkalaye, B.P., Mondal, A., Pinkas, B., Rindal, P., Yadav, A.: Weighted batched threshold encryption with applications to mempool privacy. Cryptology ePrint Archive, Paper 2025/2115 (2025), <https://eprint.iacr.org/2025/2115>
2. Agarwal, A., Babel, K., Das, S., Gilkalaye, B.P., Mondal, A., Pinkas, B., Rindal, P., Yadav, A.: Weighted batched threshold encryption with applications to mempool privacy. Cryptology ePrint Archive, Report 2025/2115 (2025), <https://eprint.iacr.org/2025/2115>
3. Agarwal, A., Babel, K., Das, S., Gilkalaye, B.P., Mondal, A., Pinkas, B., Rindal, P., Yadav, A.: Weighted batched threshold encryption with applications to mempool privacy. In: SP. pp. 2462–2481. IEEE (2026)

4. Agarwal, A., Das, S., Gilkalaye, B.P., Rindal, P., Shoup, V.: BTX: Simple and efficient batch threshold encryption. Cryptology ePrint Archive, Paper 2026/754 (2026), <https://eprint.iacr.org/2026/754>
5. Agarwal, A., Fernando, R., Pinkas, B.: Efficiently-thresholdizable batched identity based encryption, with applications. In: Kalai, Y.T., Kamara, S.F. (eds.) CRYPTO 2025, Part III. LNCS, vol. 16002, pp. 69–100. Springer, Cham (Aug 2025). [https://doi.org/10.1007/978-3-032-01881-6\\_3](https://doi.org/10.1007/978-3-032-01881-6_3)
6. Anonymous: Weighted BTE. [https://anonymous.4open.science/r/weighted\\_bte](https://anonymous.4open.science/r/weighted_bte) (2026)
7. Anza: Scheduler War. <https://schedulerwar.vercel.app/> (2026), accessed: 2026-08-06
8. Anza Technologies: solana-sdk: clock/src/lib.rs. Software repository (2026), <https://github.com/anza-xyz/solana-sdk/blob/d045b94c18f8cae8bc30eec310984030ead8d4f4/clock/src/lib.rs#L79>, commit d045b94, accessed 2026-08-02
9. Aptos Foundation: real\_dkg/rounding/mod.rs. [https://github.com/aptos-labs/aptos-core/blob/main/types/src/dkg/real\\_dkg/rounding/mod.rs](https://github.com/aptos-labs/aptos-core/blob/main/types/src/dkg/real_dkg/rounding/mod.rs)
10. Bebel, J., Ojha, D.: Ferveo: Threshold decryption for mempool privacy in BFT networks. IACR Cryptol. ePrint Arch. **2022**, 898 (2022)
11. Bebel, J., Ojha, D.: Ferveo: Threshold decryption for mempool privacy in BFT networks. Cryptology ePrint Archive, Report 2022/898 (2022), <https://eprint.iacr.org/2022/898>
12. Boneh, D., Nema, R., Roy, A., Tas, E.N.: Efficient batch threshold encryption using partial fraction techniques. Cryptology ePrint Archive, Paper 2026/674 (2026), <https://eprint.iacr.org/2026/674>
13. Boneh, D., Shoup, V.: A graduate course in applied cryptography. <https://toc.cryptobook.us/book.pdf> (2023)
14. Bormet, J., Choudhuri, A.R., Faust, S., Garg, S., Othman, H., Policharla, G.V., Qu, Z., Wang, M.: BEAST-MEV: Batched threshold encryption with silent setup for MEV prevention. Cryptology ePrint Archive, Report 2025/1419 (2025), <https://eprint.iacr.org/2025/1419>
15. Bormet, J., Faust, S., Othman, H., Qu, Z.: BEAT-MEV: Epochless approach to batched threshold encryption for MEV prevention. In: Bauer, L., Pellegrino, G. (eds.) USENIX Security 2025. pp. 3457–3476. USENIX Association (Aug 2025), <https://www.usenix.org/conference/usenixsecurity25/presentation/bormet>
16. Camenisch, J., Dubovitskaya, M., Haralambiev, K., Kohlweiss, M.: Composable and modular anonymous credentials: Definitions and practical constructions. In: ASIACRYPT (2). Lecture Notes in Computer Science, vol. 9453, pp. 262–288. Springer (2015)
17. Choudhuri, A.R., Garg, S., Piet, J., Policharla, G.V.: Mempool privacy via batched threshold encryption: Attacks and defenses. In: Balzarotti, D., Xu, W. (eds.) USENIX Security 2024. USENIX Association (Aug 2024), <https://www.usenix.org/conference/usenixsecurity24/presentation/choudhuri>
18. Choudhuri, A.R., Garg, S., Policharla, G.V., Wang, M.: Practical mempool privacy via one-time setup batched threshold encryption. In: Bauer, L., Pellegrino, G. (eds.) USENIX Security 2025. pp. 3477–3495. USENIX Association (Aug 2025), <https://www.usenix.org/conference/usenixsecurity25/presentation/choudhuri>
19. Daian, P., Goldfeder, S., Kell, T., Li, Y., Zhao, X., Bentov, I., Breidenbach, L., Juels, A.: Flash boys 2.0: Frontrunning in decentralized exchanges, miner

- extractable value, and consensus instability. In: 2020 IEEE Symposium on Security and Privacy. pp. 910–927. IEEE Computer Society Press (May 2020). <https://doi.org/10.1109/SP40000.2020.00040>
20. Das, S., Pinkas, B., Tomescu, A., Xiang, Z.: Distributed randomness using weighted VRFs. Cryptology ePrint Archive, Report 2024/198 (2024), <https://eprint.iacr.org/2024/198>
  21. Das, S., Pinkas, B., Tomescu, A., Xiang, Z.: Distributed randomness using weighted VUFs. In: Fehr, S., Fouque, P.A. (eds.) EUROCRYPT 2025, Part VII. LNCS, vol. 15607, pp. 314–344. Springer, Cham (May 2025). [https://doi.org/10.1007/978-3-031-91098-2\\_12](https://doi.org/10.1007/978-3-031-91098-2_12)
  22. Dziembowski, S., Faust, S., Luhn, J.: Shutter network: Private transactions from threshold cryptography. Cryptology ePrint Archive, Report 2024/1981 (2024), <https://eprint.iacr.org/2024/1981>
  23. Jutla, C.S., Nema, R., Roy, A.: Partial fraction techniques for cryptography. In: Daemen, J., Thomé, E. (eds.) EUROCRYPT 2026, Part V. LNCS, vol. 16545, pp. 423–453. Springer, Cham (May 2026). [https://doi.org/10.1007/978-3-032-25330-9\\_15](https://doi.org/10.1007/978-3-032-25330-9_15)
  24. Policharla, G.V.: A simple batched threshold encryption scheme. Cryptology ePrint Archive, Paper 2026/760 (2026), <https://eprint.iacr.org/2026/760>
  25. Solana core developers: Personal communication. Private communication with the authors (2026)
  26. Suegami, S., Ashizawa, S., Shibano, K.: Constant-cost batched partial decryption in threshold encryption. Cryptology ePrint Archive, Paper 2024/762 (2024), <https://eprint.iacr.org/2024/762>
  27. Supranational: blst: Multilingual BLS12-381 signature library. Software repository (2020), <https://github.com/supranational/blst>

## A Choosing total weights for the Solana chain

For our experiments, we computed per-party weights and total weight based on a snapshot of the stake distribution among Solana’s validators obtained from [7]. At the time of writing, there were 862 validator accounts with active stake, totaling about 478.4 million SOL.

Aptos implements a weighted threshold cryptographic scheme based on [20]. They implement an algorithm for assigning integer weights to parties based on stake percentages in [9]. We use Aptos’ algorithm to compute the per-party weights approximating Solana’s share distribution, choosing it because this is an algorithm with real-world usage. However, there are multiple, equally valid approaches.

To explain Aptos’ algorithm, assigning integer weights approximating each party’s stake necessarily leads to some approximation error from using integer share counts. Slightly reinterpreting Aptos’ algorithm, it takes a target threshold  $T$  and acceptable deviation  $\varepsilon$ . The algorithm computes a minimum share count such that no decryption committee with true stake fraction less than  $T - \varepsilon$  is authorized, and that any committee with true stake fraction  $T + \varepsilon$  is unauthorized. The algorithm then rounds each party’s fraction of the stake to determine the number of shares to assign to each party.

In Table 8, for  $T = 1/2$  and varying  $\varepsilon$ , we show  $W$ , the minimal total weight required to achieve error  $\varepsilon$ , as well as some statistics for each resulting assignment of weights. Specifically, we report the actual threshold  $q$ , the minimum weight required to decrypt ciphertexts. We also specify the number of validators assigned nonzero weights for each value of  $\varepsilon$  and the percentage of stake assigned to validators who receive zero shares. We also measure the quality of approximations. Letting  $\hat{p}_j$  be  $w_j/W$  be validator  $j$ 's approximate stake fraction and  $p_j$  be their true stake fraction, we report statistics on each party's absolute approximation error  $|\hat{p}_j - p_j|$ , as well as their relative error  $|\hat{p}_j - p_j|/p_j$ . Validators receiving zero weight are included in these statistics.

Table 8: Aptos-style share approximations to the Solana active-stake distribution for target threshold  $T = 2/3$  and varying allowed deviations  $\varepsilon$ . Average stake per virtual share is rounded to the nearest SOL.

| Metric                                          | Allowed deviation $\varepsilon$ |           |           |           |           |           |
|-------------------------------------------------|---------------------------------|-----------|-----------|-----------|-----------|-----------|
|                                                 | 1/4                             | 1/8       | 1/16      | 1/32      | 1/64      | 1/128     |
| Computed total weight $W$                       | 764                             | 764       | 1,580     | 3,063     | 6,489     | 12,460    |
| Minimum reconstruction weight $q$               | 404                             | 512       | 1,060     | 2,044     | 4,327     | 8,308     |
| $(T + \varepsilon)$ reconstruction boundary (%) | 63.129186                       | 75.658188 | 72.884133 | 69.791029 | 68.227607 | 67.447787 |
| Validators with nonzero weights                 | 234                             | 234       | 419       | 598       | 688       | 729       |
| Stake (%) held by validators with zero shares   | 13.3965                         | 13.3965   | 5.5137    | 1.4216    | 0.3656    | 0.1176    |
| Mean absolute error (pp)                        | 0.032983                        | 0.032983  | 0.015630  | 0.007260  | 0.003622  | 0.001814  |
| Median absolute error (pp)                      | 0.023706                        | 0.023706  | 0.016225  | 0.007337  | 0.003650  | 0.001768  |
| Maximum absolute error (pp)                     | 0.452110                        | 0.452110  | 0.078842  | 0.019015  | 0.007689  | 0.004087  |
| Median relative error (%)                       | 100.00                          | 100.00    | 100.00    | 25.98     | 13.14     | 5.90      |

We choose the weight distributions with  $\varepsilon = 1/8, 1/16, 1/32, 1/64$  for our experiments. The resulting total share counts are comparable to those used by [20]'s experiments. Furthermore, at  $\varepsilon = 1/32, 1/64$ , the high and low end reconstruction thresholds come within a few percentage points of the true threshold, and the various error statistics drop to a few basis points. Our reimplementations of Aptos' share assignment rule and generated allocations are available in the `scripts` directory at [6].

**Setup**( $1^\lambda, 1^\ell, (\Omega_j)_{j \in [N]}, 1^t, 1^W$ )  $\rightarrow$  (ek, sk, vk, dk):

- ▷  $\text{crs}_{\Pi_{\text{DL}}} \leftarrow \Pi_{\text{DL}}.\text{Setup}(1^\lambda)$ .
- ▷ Sample  $z \leftarrow \mathbb{F}_p$  and a uniformly random polynomial  $Z(X) \in \mathbb{F}_p[X]$  of degree at most  $t$  such that  $Z(0) = z$ . Set  $z_w := Z(w)$  for  $w \in \Omega_{[N]}$ .
- ▷ Fix distinct public indices  $\alpha_1, \dots, \alpha_\ell \in \mathbb{F}_p \setminus \{0, -1\}$ . These values are included in the public parameters. Sample  $\beta_{-1}, \beta_0 \leftarrow \mathbb{F}_p$  subject to  $\beta_0 \neq 0$  and  $\beta_{-1} + \beta_0 \neq 0$ . For every  $j \in [N]$ , sample a hidden shift  $\rho_j$  such that every  $\rho_j + \alpha_i$  is nonzero and the values  $\rho_j + \alpha_i$ , over  $(j, i) \in [N] \times [\ell]$ , are distinct. Define

$$g_{j,i} := \frac{1}{\rho_j + \alpha_i}, \quad u_{j,i} := g_{j,i}^2 + \frac{\beta_{-1}}{\alpha_i + 1} + \frac{\beta_0}{\alpha_i}$$

for every  $j \in [N]$  and  $i \in [\ell]$ .

- ▷ Let  $\text{ek} := [z\beta_0]_T$ , and let  $\text{sk}_j := \rho_j$  and  $\text{vk}_j := (\text{vk}_{j,i} := [g_{j,i}]_2)_{i \in [\ell]}$  denote party  $j$ 's secret and verification keys.
- ▷ Define  $\text{dk}[0] := [z(\beta_{-1} + \beta_0)]_2$ , and, for every  $w \in \Omega_{[N]}$  and  $i \in [\ell]$ , define

$$\text{dk}_{w,i}[1] := [z_w g_{\text{own}(w),i}]_2, \quad \text{dk}_{w,i}[2] := [z_w u_{\text{own}(w),i}]_2.$$

- ▷ **Output**

$$\left( \text{ek}, (\text{sk}_j)_{j \in [N]}, (\text{vk}_j)_{j \in [N]}, \text{dk} := \left( \text{dk}[0], (\text{dk}_{w,i}[1], \text{dk}_{w,i}[2])_{w \in \Omega_{[N]}, i \in [\ell]} \right) \right).$$

**Enc**(ek,  $m$ )  $\rightarrow$  ct:

- ▷ Given  $m \in \mathbb{G}_T$ , sample  $r \leftarrow \mathbb{F}_p$ , and set  $\text{ct}[1] := [r]_1$  and  $\text{ct}[2] := m + r \cdot \text{ek}$ .
- ▷ Obtain  $\pi \leftarrow \Pi_{\text{DL}}.\text{Prove}((\text{ct}[1], \text{ct}[2]); r)$  and output  $\text{ct} := (\text{ct}[1], \text{ct}[2], \pi)$ .

**PreDec**( $\text{sk}_j, (\text{ct}_i)_{i \in [\ell]}$ )  $\rightarrow$  sbk <sub>$j$</sub> :

- ▷ Parse  $\text{ct}_i = (\text{ct}_i[1], \text{ct}_i[2], \pi_i)$  for every  $i \in [\ell]$ . If  $\Pi_{\text{DL}}.\text{Verify}((\text{ct}_i[1], \text{ct}_i[2]), \pi_i) = 0$  for any  $i \in [\ell]$ , return  $\perp$ .
- ▷ Otherwise, return  $\text{sbk}_j := \sum_{i=1}^{\ell} \frac{1}{\rho_j + \alpha_i} \text{ct}_i[1] \in \mathbb{G}_1$ .

**Dec**(dk, sbk,  $(\text{ct}_i)_{i \in [\ell]}$ )  $\rightarrow$  ( $m_i$ ) <sub>$i \in [\ell]$</sub> :

- ▷ Verify each submitted pre-decryption key by checking

$$e(\text{sbk}_j, [1]_2) = \sum_{i=1}^{\ell} e(\text{ct}_i[1], \text{vk}_{j,i}).$$

Discard the keys which do not pass verification. Let  $T'$  be an authorized set of accepted pre-decryption keys, and return  $\perp$  if no such set remains.

- ▷ For every  $j \in T'$  and  $i \in [\ell]$ , compute

$$\begin{aligned} L_{j,i} := & e \left( \text{sbk}_j - \sum_{\substack{k \in [\ell] \\ k \neq i}} \frac{\text{ct}_k[1]}{\alpha_k - \alpha_i}, \sum_{w \in \Omega_j} \lambda_{T',w} \text{dk}_{w,i}[1] \right) \\ & + \sum_{\substack{k \in [\ell] \\ k \neq i}} e \left( \frac{\text{ct}_k[1]}{\alpha_k - \alpha_i}, \sum_{w \in \Omega_j} \lambda_{T',w} \text{dk}_{w,k}[1] \right). \end{aligned}$$

- ▷ For every  $i \in [\ell]$ , output  $m_i := \text{ct}_i[2] - R_i$ , where

$$R_i := \alpha_i \left( (\alpha_i + 1) \left( e \left( \text{ct}_i[1], \sum_{w \in \Omega_{T'}} \lambda_{T',w} \text{dk}_{w,i}[2] \right) - \sum_{j \in T'} L_{j,i} \right) - e(\text{ct}_i[1], \text{dk}[0]) \right).$$

**Construction 5:** Partial fraction-based weighted BTE construction

Table 3: Storage/communication costs for Construction 5.  $N$  is the number of real parties,  $W = \sum_{j=1}^N w_j$  is the total virtual weight,  $\tau = |T|$  is the number of parties in the accepted set, and  $\ell$  is the padded batch size. We exclude costs related to the NIZK and descriptions of bilinear groups.

| <b>Storage costs</b>                                                  |                                                     |
|-----------------------------------------------------------------------|-----------------------------------------------------|
| Quantity                                                              | Size                                                |
| Encryption key $\mathbf{ek}$                                          | $ \mathbb{G}_T $                                    |
| Party secret key $\mathbf{sk}_j$                                      | $\ell \mathbb{F}_p $                                |
| Party verification key $\mathbf{vk}_j$                                | $\ell \mathbb{G}_2 $                                |
| All parties' verification keys                                        | $N\ell \mathbb{G}_2 $                               |
| Core decryption material $(D_{w,i}, U_{w,i}, V)$                      | $(2W\ell + 1) \mathbb{G}_2 $                        |
| Public decryption key $\mathbf{dk}$ , including all verification keys | $((2W + N)\ell + 1) \mathbb{G}_2 $                  |
| Non-NIZK CRS / total public parameters $(\mathbf{ek}, \mathbf{dk})$   | $((2W + N)\ell + 1) \mathbb{G}_2  +  \mathbb{G}_T $ |
| Ciphertext, excluding $\pi_{\text{DL}}$                               | $ \mathbb{G}_1  +  \mathbb{G}_T $                   |
| Padded batch of ciphertexts, excluding proofs                         | $\ell \mathbb{G}_1  + \ell \mathbb{G}_T $           |
| One party's pre-decryption $\sigma_j$                                 | $ \mathbb{G}_1 $                                    |
| Pre-decryptions from an accepted set $T$                              | $\tau \mathbb{G}_1 $                                |
| Decrypted message batch                                               | $\ell \mathbb{G}_T $                                |

Table 4: Computation costs for Construction 5. For  $a \in \{1, 2, T\}$ ,  $\text{SM}_a$  is a scalar multiplication in  $\mathbb{G}_a$ , and  $\text{MSM}_a(q)$  is a multi-scalar multiplication of size  $q$  in  $\mathbb{G}_a$ . Group additions are not counted.  $\text{P}$  denotes a pairing,  $\text{MP}(q)$  denotes a multi-pairing of size  $q$ , and  $\text{CauchyFFT}_a(q)$  denotes a size  $q$  “Cauchy FFT” in  $\mathbb{G}_a$ , as previously described.  $W_T$  is the total weight of parties decrypting a ciphertext batch.

| <b>Computational costs</b>                                      |                                                                                                                                                                                                                      |
|-----------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Operation                                                       | Cost                                                                                                                                                                                                                 |
| Enc, one ciphertext                                             | $1 \text{ SM}_1 + 1 \text{ SM}_T$                                                                                                                                                                                    |
| PreDec $_j$ , one padded batch                                  | $1 \text{ MSM}_1(\ell)$                                                                                                                                                                                              |
| Verify one submitted pre-decryption $\sigma_j$                  | $1 \text{ MP}(\ell + 1)$                                                                                                                                                                                             |
| Dec, cached displayed form                                      | $\ell(\tau + 1)\text{P} + \ell \text{ MP}(\tau + 1) + \ell \text{ MSM}_1(\ell - 1) + \sum_{j \in T} \ell \text{ MSM}_2(w_j) + \ell \text{ MSM}_2(W_T) + \tau \ell \text{ MSM}_T(\ell - 1) + \ell \text{ MSM}_T(2)$   |
| Dec, cached and bilinearly aggregated                           | $2\ell\text{P} + \ell \text{ MP}(\tau) + \ell \text{ MSM}_1(\ell - 1) + \sum_{j \in T} \ell \text{ MSM}_2(w_j) + \ell \text{ MSM}_2(W_T) + \ell \text{ MSM}_2(2) + \ell \text{ MSM}_T(\ell - 1) + \ell \text{ SM}_T$ |
| Dec, cached/aggregated pairings and FFT-accelerated Cauchy sums | $2\ell\text{P} + \ell \text{ MP}(\tau) + \text{CauchyFFT}_1(\ell) + \sum_{j \in T} \ell \text{ MSM}_2(w_j) + \ell \text{ MSM}_2(W_T) + \ell \text{ MSM}_2(2) + \text{CauchyFFT}_T(\ell) + \ell \text{ SM}_T$         |

| $W$   | PartialDecrypt<br>(ms/item) |       | Validate sbk's<br>(ms/item) |       | Decrypt Precomp<br>(ms/item) |        | Decrypt Open<br>(ms/item) |       | Decrypt Total<br>(ms) |       |
|-------|-----------------------------|-------|-----------------------------|-------|------------------------------|--------|---------------------------|-------|-----------------------|-------|
|       | WBTX                        | WPFE  | WBTX                        | WPFE  | WBTX                         | WPFE   | WBTX                      | WPFE  | WBTX                  | WPFE  |
| 764   | 0.086                       | 0.087 | 0.326                       | 0.345 | 4.597                        | 4.031  | 0.0549                    | 0.405 | 170.6                 | 155.8 |
| 1,580 | 0.079                       | 0.082 | 0.291                       | 0.308 | 7.777                        | 7.612  | 0.468                     | 0.510 | 277.9                 | 276.0 |
| 3,060 | 0.076                       | 0.078 | 0.289                       | 0.288 | 12.492                       | 12.804 | 0.485                     | 0.523 | 429.9                 | 442.4 |
| 6,486 | 0.078                       | 0.080 | 0.289                       | 0.289 | 22.867                       | 21.515 | 0.488                     | 0.529 | 762.0                 | 721.5 |

Table 5: Mean running times of decryption phases while varying total weight. Batch size  $\ell$  is fixed at 32 for these experiments.

| $W$ | PartialDecrypt<br>(ms/item) |       | Validate sbk's<br>(ms/item) |       | Decrypt Precomp.<br>(ms/item) |       | Decrypt Opening<br>(ms/item) |       | Decrypt Total<br>(ms) |        |
|-----|-----------------------------|-------|-----------------------------|-------|-------------------------------|-------|------------------------------|-------|-----------------------|--------|
|     | WBTX                        | WBTE  | WBTX                        | WBTE  | WBTX                          | WBTE  | WBTX                         | WBTE  | WBTX                  | WBTE   |
| 32  | 0.081                       | 0.083 | 0.294                       | 0.295 | 7.816                         | 7.401 | 0.469                        | 0.500 | 279.4                 | 268.7  |
| 64  | 0.177                       | 0.129 | 0.615                       | 0.530 | 10.93                         | 7.340 | 0.572                        | 0.489 | 766.1                 | 528.4  |
| 128 | 0.208                       | 0.205 | 1.071                       | 1.002 | 7.399                         | 7.521 | 0.436                        | 0.455 | 1051.4                | 1070.9 |
| 256 | 0.345                       | 0.346 | 2.105                       | 2.007 | 7.221                         | 6.971 | 0.439                        | 0.496 | 2054.1                | 2008.4 |

Table 6: Mean running time of decryption phases while varying batch size. The total weight is fixed at 1,580 for these experiments.

| $W$   | $\ell$ | Encrypt<br>(ms/item) |       | Decryption Key<br>Size (kB) |       | Verification Key<br>Size (kB) |      | Decryption Key<br>Size Total (kB) |       |
|-------|--------|----------------------|-------|-----------------------------|-------|-------------------------------|------|-----------------------------------|-------|
|       |        | WBTX                 | WBTE  | WBTX                        | WBTE  | WBTX                          | WBTE | WBTX                              | WBTE  |
| 764   | 32     | 0.059                | 0.061 | 4512                        | 4584  | 702                           | 702  | 5214                              | 5286  |
| 1,580 | 32     | 0.059                | 0.058 | 9308                        | 9456  | 1257                          | 1257 | 10585                             | 10713 |
| 3,063 | 32     | 0.066                | 0.060 | 18073                       | 18360 | 1794                          | 1794 | 19867                             | 20154 |
| 6,489 | 32     | 0.058                | 0.058 | 38308                       | 38916 | 2064                          | 2064 | 40372                             | 40980 |
| 1,580 | 32     | 0.060                | 0.058 | 9308                        | 9456  | 1257                          | 1257 | 10565                             | 10713 |
| 1,580 | 64     | 0.057                | 0.056 | 18764                       | 18912 | 2514                          | 2514 | 21278                             | 21426 |
| 1,580 | 128    | 0.050                | 0.058 | 37676                       | 37824 | 5028                          | 0.00 | 42704                             | 0.0   |
| 1,580 | 256    | 0.049                | 0.000 | 75500                       | 0.00  | 10056                         | 0.00 | 85556                             | 0.0   |

Table 7: Encryption cost and key sizes.